Whiteout AI: An Accuracy and Efficiency Benchmark of the Semantic Policy Engine for Enterprise AI – Extended Technical Report

Alexander Flowers

Abstract—Enterprise adoption of generative artificial intelligence creates a tension: organizations must enable AI productivity while preventing data leakage, policy violations, and misuse. Pattern-based data loss prevention (DLP) misclassifies meaning-level content, and cloud content-safety APIs offer only fixed, vendor-defined taxonomies. We present a 100,000-prompt accuracy and efficiency benchmark of the deployed semantic compliance engine of Whiteout AI: an open-weights, approximately 20-billion-parameter, US-developed large language model (LLM) served locally on a single GPU under the organization's control, which reads the full administrator-authored ruleset of 54 deployed policies in its system turn, answers on a forced final channel with schema-constrained decoding, runs a deterministic pre-analysis of identifiers and obfuscation before the model, escalates about 3% of prompts to full reasoning under a hybrid rule, and fails closed on any incomplete verdict. Across 8 categories the engine achieves 96.53% raw accuracy and 96.78% corrected accuracy (95% confidence interval [96.65%, 96.95%]), where corrected accuracy credits only those misses that independent readers adjudicated as label errors against the deployed rule text; it blocks 96.43% of violations (recall), allows 96.59% of pass-expected prompts (specificity), and passes 99.89% of 36,416 benign everyday prompts. Sustained throughput is 9.64 prompts/s on two engine instances, with a sub-two-second median latency (1.10 s) for allowed prompts: 2.4× the throughput of full chain-of-thought reasoning, within 0.14 corrected points of its accuracy. We re-examine the Admin Autonomy Property, that enforcement follows administrator configuration alone, and find the forced channel enforcing switched-off content under neighbouring rules; a deterministic post-resolution guard restores the property for identifier-class policies (191 of 197 bare-card-number prompts allowed with the card-number policy off), with semantic policies remaining an open item (September 2026).

Index Terms—AI governance, data loss prevention, large language models, semantic compliance, LLM-as-judge, enterprise security, locally hosted inference, structured decoding, fail-closed enforcement, reproducibility.

I. Introduction

Enterprise adoption of large language models (LLMs) has crossed from experiment to default: code assistants are embedded in development pipelines, conversational agents ship inside employee productivity suites, and third-party LLM tools are reached from browsers, desktops, and integrated development environments. With that adoption comes a compliance tension: every prompt an employee types into a third-party AI tool is a candidate data disclosure, and many such disclosures violate regulation — the General Data Protection Regulation (GDPR), HIPAA — or internal policy (trade secrets, attorney-client privilege, unreleased earnings). The European Data Protection Board opinion on AI-model processing [1] makes explicit what the memorization literature [2], [3] had already implied: sending prompts containing personal data to a cloud LLM is processing under GDPR and requires a lawful basis.

The canonical control is enterprise data loss prevention (DLP). Pattern-based DLP suffers the semantic gap [4], [5]: a regular expression that recognizes the format of a Social Security Number (SSN) flags both the SSN in a violation prompt and the documentation of the SSN format in a safe prompt, and it has nothing to say about the confidential roadmap, the unreleased contract term, or the clinical narrative that contains no pattern at all. The false-positive rate this generates is large enough that administrators routinely disable rules [6]. Cloud AI content-safety APIs (Azure AI Content Safety, OpenAI Moderation, Google Perspective) avoid the semantic gap — they reason with models — but they offer only fixed, vendor-defined taxonomies (hate, violence, sexual content, self-harm) that are disjoint from enterprise compliance categories, cannot be extended with administrator-authored natural-language rules, and require the prompt to leave the organization. Developer-facing guardrails frameworks (NeMo Guardrails, Guardrails AI) require code integration and presume the enterprise controls the LLM call site, which is not the case for third-party AI tools.

We report a benchmark of a different architecture, deployed in production as the compliance engine of Whiteout AI, the enterprise AI governance product of Groovy Security. The engine is an open-weights, approximately 20-billion-parameter, US-developed mixture-of-experts model post-trained for policy-conditioned classification, served locally on a single GPU under the organization's control; prompts do not leave the organization's boundary for the compliance judgment. The full administrator-authored ruleset — 54 deployed policies written as natural-language rules with inline examples and exception clauses — occupies the system turn, the user prompt follows, and

Alexander Flowers is with Groovy Security, Inc., St. George, UT, USA, and with Groovy Security, Ltd., Dublin, Ireland. E-mail: alex@groovysec.com.

Extended technical report, September 2026 edition. Includes the full efficiency analysis, the error analysis and adjudication protocol, configuration comparisons, expanded related work, and a reproducibility appendix.

the inference server caches the roughly 19,800-token policy prefix so that each verdict costs only the prompt's own tokens. The verdict is produced on a forced final channel: the model is prompted directly into its answer with no room for chain-of-thought, and grammar-enforced structured decoding constrains the output to a schema of status, reason, cited rules, and optional rewrite, at roughly 71 generated tokens per verdict rather than the roughly 249 that full reasoning produces.

Two mechanisms surround that judgment. Before the model runs, a deterministic pre-analysis pass detects identifier shapes (emails, IP and MAC addresses, UUIDs, GPS pairs, serial-shaped tokens, authentication codes) and mechanically reverses obfuscation (homoglyph and unicode folding, separator stripping, vertical layouts, base64 and hex decoding, leetspeak, spelled-out digits, announced reversals, split halves); its findings are appended to the evaluation input as data, never as instructions, and never override the verdict. After the model answers, a hybrid escalation re-judges once, with full reasoning, any compliant verdict that coincides with a detected value shape or a semantic-risk signal, and any non-compliant verdict whose own reason admits that the content is masked; about 3% of prompts escalate, and blocked verdicts are otherwise never escalated, so the block path stays at forced-channel speed. A verdict that hits its token ceiling or cannot be parsed is held — blocked with an explicit "compliance incomplete" reason — never allowed.

a) Contributions.: We make four contributions.

1) A deployed pure-semantic compliance architecture. We describe and benchmark a production engine that judges every prompt against the full natural-language ruleset with no regular-expression, keyword, or deterministic-override verdict layer, produces schema-constrained verdicts on a forced final channel, and escalates about 3% of prompts to full reasoning under a hybrid rule (Sections III and V).
2) A 100,000-prompt benchmark with a corrected-accuracy protocol. The corpus spans 8 categories (personally identifiable information (PII), protected health information (PHI), GDPR, Legal, Code, Confidential, Security, and Finance), 70 benchmark test facets mapping many-to-one onto the 54 deployed policies, six bands (core, long, adversarial, mixed, multilingual, and benign), and 13 obfuscation techniques, generated deterministically and validated before scoring (Section IV). The engine achieves 96.53% raw accuracy and 96.78% corrected accuracy (95% confidence interval (CI) [96.65%, 96.95%]), blocks 96.43% of violations, allows 96.59% of pass-expected prompts, and passes 99.89% of 36,416 benign prompts (Section VII-A). Corrected accuracy follows an adjudication protocol in which eight independent readers judge a random sample of misses against the deployed rule text; a miss is credited only when the rule verdict disagrees with the label, and arguable calls are not credited.
3) Efficiency at interactive speed. The deployed configuration sustains 9.64 prompts/s on two engine instances at 48 concurrent requests, completing the corpus in about 2.9 hours, with a median latency of 1.10 s for allowed prompts and 2.69 s for blocked prompts at 16 concurrent requests (Sections VII-H and VIII). Against full chain-of-thought reasoning on the same corpus and the same scoring, the hybrid delivers 2.4× the throughput within 0.14 corrected points of accuracy (Section X).
4) Fail-closed enforcement, and a measured account of the Admin Autonomy Property. We show that an incomplete or unparseable verdict is held rather than allowed, and verify on the corpus that 0 of 100,000 prompts fall through to fail-open (Sections V-C and VII). We define the Admin Autonomy Property — enforcement follows administrator configuration alone — and measure it honestly: the forced channel enforces switched-off content under neighbouring rules and cannot be instructed out of it, so we restore the property mechanically for identifier-class policies with a post-resolution guard (191 of 197 bare-card-number prompts allowed with the card-number policy off) and report the semantic residue (Sections VII-G and XI-G).

b) Artifact.: The corpus generators, the corpus validator and its committed manifest, the fixed seeds, the resumable concurrent runner, the adjudication protocol with reader outputs, and the retest harness with its 800 seeded control prompts are versioned together. The corpus is released publicly on Hugging Face under Apache 2.0 (https://huggingface.co/datasets/ShmalexFlow/enterprise-ai-prompt-compliance-100k); the generators, runner, adjudication protocol, reader outputs, and retest harness are available to partners on request. Appendix A describes them.

c) Outline.: Section II surveys prior work in DLP, cloud content-safety APIs, LLM guardrails, LLM-as-judge methods, prompt injection, and local inference. Section III describes the compliance engine and Section IV the benchmark. Section V details the architecture and Section VI the policy-rule format. Section VII reports accuracy and Section VIII throughput and latency. Section IX analyzes the errors, and Section X compares engine configurations on the same corpus. Section XI interprets the results and Section XI-G states limitations. We conclude in Section XII; Appendix A is the reproducibility appendix.

II. Related Work

The problem of evaluating user-authored content against an organization's policy is attacked from four distinct technical traditions: enterprise Data Loss Prevention (DLP), cloud AI content-safety APIs, LLM guardrails frameworks, and the LLM-as-judge literature. A fifth adjacent tradition — prompt-injection and jailbreak research — bounds the threat model. A sixth — local-inference and data-residency work — motivates the architectural choice of locally-hosted inference. We survey each and, for each,

name the property it supplies and the property it lacks relative to the compliance engine evaluated in this report. The full gap synthesis is in Section II-G at the end of the section.

A. Data Loss Prevention

Commercial enterprise DLP is dominated by pattern-based detection: regular expressions, keyword lists, document fingerprinting, exact-data matching, and machine-learning-assisted classifiers layered on top of them. Shabtai et al. [4] establish the foundational four-pillar taxonomy of detection (regex/keyword, partial fingerprinting, exact match, statistical / ML classifier) and note that regex plus keyword matching dominates because they are transparent to administrators, at the cost of substantial false-positive volume on free-form text. Alneyadi et al. [5] update the taxonomy and articulate the semantic gap problem most explicitly: a pattern detector cannot distinguish a benign discussion of a credit-card format from an actual credit-card number appearing in context. Costante et al. [6] observe that the resulting false-positive volume drives administrators to disable rules — a concrete operational failure mode of rule-based DLP in production. Hauer [7] surveys the DLP market and confirms the three-pillar architecture (network, endpoint, storage) that dominates commercial vendor offerings. Hart et al. [8] demonstrate that text classification on narrow enterprise-email categories can reach $F_1 \in [0.90, 0.95]$, but only with per-category training corpora that most enterprises cannot produce for the breadth of policies they need.

Three properties recur in this literature. First, there is no community-standard benchmark for enterprise DLP accuracy: Shabtai et al. and Alneyadi et al. both call out the absence of a shared evaluation corpus, and vendor-published accuracy numbers are not independently verifiable. Second, the commercial Leader set (Symantec / Broadcom, Forcepoint, Microsoft Purview; see Gartner's longitudinal Magic-Quadrant coverage) is stable: no DLP vendor has shipped a primary-classifier architecture in which a single LLM reads the full natural-language policy ruleset. Third, where ML is introduced it is layered atop pattern rules rather than replacing them. The engine evaluated in this report inverts that layering: deterministic pre-analysis only annotates the input, and the verdict is the model's reading of the full natural-language ruleset — evaluated at the 100,000-prompt scale this literature has long noted is missing.

B. Cloud AI Content-Safety APIs

The nearest "AI-based content compliance" product family is cloud-hosted content-safety APIs. Three specific APIs anchor the category.

Markov et al. [9] describe OpenAI's Moderation endpoint (omni-moderation-latest), with a taxonomy covering hate, harassment, self-harm, sexual content, and violence (each with sub-categories), reporting F_1 in the 0.7–0.8 range on their internal evaluation set. Lees et al. [10] describe the current Charformer-based generation of Google Jigsaw's Perspective API, with AUC around 0.97 on TOXICITY across many languages and ongoing concerns around calibration and bias. Sap et al. [11] show that Perspective and similar classifiers systematically over-flag African-American English and similar dialectal variants; independent replication confirms the finding. Microsoft's Azure AI Content Safety targets the same four macro-categories (hate, sexual, violence, self-harm) plus prompt-shield and groundedness features; it is cloud-only for the primary SKU, and on-prem only via containerized deployment in enterprise SKUs.

Inan et al. [12] shift the category toward open-weights with Llama Guard, an LLM-based input/output safeguard. Llama Guard is the closest existing analog to the engine evaluated here — an LLM-based moderator an enterprise can self-host — but its taxonomy remains community-defined (MLCommons AI Safety Working Group categories) and administrators cannot extend it with organization-specific natural-language rules without retraining. On ToxicChat and OpenAIMod, Inan et al. report Moderation API macro-$F_1 \approx 0.76$ and Llama Guard 2 typically ≥ 0.80, establishing that frontier-lab content- safety APIs top out well below the accuracy numbers this paper reports — on a different, simpler task.

Two properties recur. First, every API in this category targets general-purpose harmful content, not enterprise- compliance categories (PHI, GDPR, trade secret, legal privilege, confidential roadmap). There is no administrator-configurable natural-language-rule interface; categories are fixed and integrators adjust thresholds. Second, cloud delivery is the default, with data-residency implications that motivate Section II-F.

C. LLM Guardrails Frameworks

Developer-facing guardrails frameworks are a different shape of tool. NVIDIA NeMo Guardrails [13] provides a DSL (Colang) for expressing topic-level and tool-use guardrails programmatically; Guardrails AI [14] provides RAIL (an XML-based schema) plus a runtime for output validation. Dong et al. [15] survey the space. Kang et al. [16] demonstrate that the programmatic layer itself is susceptible to bypass through standard prompt-engineering attacks.

These frameworks share a precondition that does not hold for the enterprise-AI-governance problem: they presume the enterprise controls the LLM call site. For internal LLM-backed applications this is true; for third-party AI tools an employee opens in a browser (ChatGPT, Claude, Gemini, Copilot Chat) it is not. The guardrails operate on outputs from an LLM the enterprise itself invoked; they do not intercept prompts on their way to a provider the enterprise does not run. Whiteout AI's five enforcement surfaces (Section V-D) address this architectural mismatch by intercepting at the user boundary rather than the LLM call site.

A second property: these frameworks require code changes and DSL authorship. The administrator-authored natural-language policy surface evaluated in this paper is available without DSL expertise.

D. LLM-as-Judge

The LLM-as-judge literature is the nearest methodological analog. Zheng et al. [17] established that GPT-4-class judges achieve agreement with crowd humans on open-ended response ranking comparable to human-human agreement on MT-Bench and Chatbot Arena, and they systematically studied judge biases (position, verbosity, self-preference). Bai et al. [18] present Constitutional AI, in which an LLM judges its own outputs against a natural-language constitution; this is the closest architectural parallel to the administrator-authored natural-language-rule surface in this paper, with the difference that Constitutional AI is a training-time self-critique technique whereas our setup is an inference-time compliance evaluation. Chiang and Lee [19] offer an early direct-substitution study with caveats. Liu et al. [20] present G-Eval, an evaluator framework using form-filling chain-of-thought.

The documented biases are the central field-level concern. Wang et al. [21] document position bias: LLM judges prefer responses shown first or last in a pair. Dubois et al. [22] document length bias: LLM judges prefer longer responses. Li et al. [23] catalog further cognitive biases. Gu et al. [24] synthesize the landscape in a recent comprehensive survey. Most of this literature uses frontier-scale closed models, and the reliability of mid-scale open-weights models (an open-weights model of roughly 20 billion parameters, as in our engine) as enterprise-compliance judges is under-evaluated.

Two reads follow. First, the 96.78% corrected accuracy reported here for an open-weights model of roughly 20 billion parameters is an empirical datapoint for the under-evaluated mid-scale open-weights judge regime. Second, the biases catalogued above are ranking phenomena: position bias and length bias arise when a judge compares candidate responses, whereas compliance judgment is binary classification of a single prompt under a fixed ruleset with a schema-constrained output, so the mechanism that produces them has no pair of responses to act on. The accuracy gradient the benchmark does show across prompt length (Section VII-E) tracks the violation-heavy construction of the long band — buried payloads, split payloads, decoy safes — rather than a judge preference for length.

E. Prompt Injection and Jailbreaks

Prompt-injection attacks [25], [26] and jailbreak attacks [27], [28] target an LLM's behavior via adversarial input. Defenses and benchmarks — baseline filtering [29], HarmBench [30], JailbreakBench [31] — remain an open arms race.

This literature scopes a threat model distinct from the one evaluated in this report. The compliance engine's threat model covers users who obfuscate content a rule forbids — the adversarial band of the benchmark, thirteen techniques from homoglyphs to base64 — but not adaptive attackers actively targeting the engine's own system prompt, which is a separate problem that requires dedicated evaluation we have not run. We flag this as a limitation (Section XI-G) and do not claim prompt-injection robustness.

F. Local Inference and Data Residency

Running LLM inference on organization-controlled infrastructure is motivated by two literatures. The memorization and extraction literature — Carlini et al. [2] on training-data extraction from language models, Nasr et al. [3] on production-model extraction, Pan et al. [32] on embedding-inversion attacks against cloud APIs — establishes that prompts sent to third- party models are not guaranteed to remain confidential. The regulatory literature — EDPB Opinion 28/2024 [1] — holds that sending prompts containing personal data to a cloud LLM constitutes processing under GDPR and requires a lawful basis. The Italian Garante's 2023 temporary ban on ChatGPT under similar reasoning is an earlier datapoint.

Published technical reports on open-weights model families, read against MMLU, the benchmark established by Hendrycks et al. [33], document that open-weights models reach parity with proprietary frontier models for many enterprise tasks — the empirical grounding for "an open-weights model served locally is capable enough for this job."

G. What is missing

Synthesizing the six literatures above, four gaps motivate this report. No prior work combines (i) a single LLM reading the full administrator-authored natural-language policy ruleset as the primary compliance classifier; (ii) a formalized and empirically validated property that administrator configuration is the sole source of enforcement (Section III-I); (iii) a verdict path — forced final channel, deterministic pre-analysis, selective escalation — that holds a sub-2-second median latency for allowed prompts while landing within 0.14 corrected points of full chain-of-thought reasoning on the same 100,000 prompts; and (iv) a fail-closed verdict rule validated on that corpus, with 0 of 100,000 prompts falling through to fail-open. We present the evidence for each in Sections III–XI.

A probable objection is that the report does not run a head-to-head comparison against cloud content-safety APIs or Llama Guard. Such a comparison is not tractable as apples-to- apples: cloud APIs and Llama Guard do not support natural-language-rule policy configuration, so the comparison requires reformulating the 54 natural-language rules into a fixed-taxonomy threshold schedule, which is a different system rather than the same one. We scope our claim accordingly: the semantic-compliance architecture works at production scale with the empirical numbers reported here, with the caveat in Section XI-G that a

Figure 1 — System architecture. Five enforcement surfaces (browser, desktop, IDE, infrastructure, admin dashboard) forward prompts to the locally-hosted compliance engine: deterministic pre-analysis, a forced-channel verdict under the cached policy prefix, a single full-reasoning re-judge on escalation triggers, and fail-closed completion handling.

Fig. 1. Whiteout AI architecture.

cross-architecture comparative benchmark on identical policy sets is an open problem. The corpus is available to partners on request, with a public release planned.

III. The Compliance Engine

We describe the deployed compliance engine as it runs today, then the construction of the benchmark that evaluates it.

A. Platform Overview

Whiteout AI intercepts user-authored prompts at five enforcement surfaces before they leave the organization's boundary: a Chromium browser extension for web-based AI tools, a desktop guard for native AI applications (macOS, Windows), an IDE extension (VS Code, JetBrains) for coding assistants, an infrastructure agent deployed as a sidecar (Kubernetes, EC2, Docker) for server-side AI API calls, and an administrator dashboard for configuration, audit, and policy testing. Each surface forwards the intercepted prompt to the locally-hosted compliance engine; the engine returns a structured verdict (status, reason, cited rules, and an optional compliant rewrite); the surface then allows the prompt, blocks it, or offers the rewrite in its place.

B. Model and Serving

The compliance engine is an open-weights, US-developed, ∼20-billion-parameter mixture-of-experts large language model (LLM) post-trained for policy-conditioned classification, served locally on a single graphics processing unit (GPU) under the organization's control by the inference server. Inference is local: a prompt never leaves the organization's boundary for the compliance judgment.

Every evaluation places the full administrator-authored ruleset — all 54 deployed policies, written as natural-language rules with inline examples and exception clauses — in the system turn; the user content follows. The policy prefix is approximately 19,800 tokens, and the inference server prefix-caches it, so each verdict costs only the prompt's own tokens. One consequence governs the efficiency results of Section VIII: latency per verdict scales with the number of output tokens, not with input length.

C. Forced-Channel Verdicts

The engine produces every verdict on a forced final channel: the model is prompted directly into its answer channel, with no room for chain-of-thought, and the output is constrained to a fixed schema — status, reason, cited rules, and optional rewrite — by grammar-enforced structured decoding. A verdict is therefore approximately 71 output tokens, against approximately 249 when the same model is allowed full reasoning: 3.5× fewer generated tokens. The parser accepts only completions that conform to the schema; Section III-F gives the disposition of a completion it cannot use.

D. Deterministic Pre-Analysis

Before the model runs, two deterministic components analyse the full content block. An identifier pre-detector finds identifier-shaped values — emails, IP addresses, MAC addresses, UUIDs, GPS coordinate pairs, serial-shaped tokens, and authentication codes — and excludes placeholders. A canonicalizer mechanically reverses obfuscation: Unicode and homoglyph folding, separator stripping, vertical layouts, base64 and hex decoding, leetspeak, spelled-out digits, announced reversals, and split halves. The findings of both components are appended to the evaluation input as data, never as instructions: they inform the model's judgment and the escalation decision below, but the verdict remains the model's, made against the rule text.

E. Hybrid Escalation

Most verdicts are final on the forced channel. A forced-channel COMPLIANT verdict that coincides with a pre-analysis value shape, or with a semantic-risk signal (schedule, code, formula, money-plan, roster, deal, negotiation, HR-review, litigation, and clinical-context shapes), is re-judged once with full reasoning, and that second verdict wins. A NON-COMPLIANT verdict whose own reason admits that the content is masked, with no value detected by pre-analysis, is likewise re-judged once. Blocked verdicts are otherwise never escalated, so the block path stays at forced-channel speed. About 3% of prompts escalate; each escalation costs one additional full-reasoning call, typically 4–10 s.

F. Fail-Closed Completion Handling

A completion that hits the token ceiling, or that the parser cannot use, is held: the prompt is blocked with an explicit "compliance incomplete" reason, and it is never allowed. Only a genuinely unreachable engine — an outage, not a verdict — follows the organization's configured fail-open or fail-closed setting. We verified this behaviour on the benchmark: after the fix described in Section VII, 0 of 100,000 prompts fell through to fail-open.

G. Chunking, Rewrite, and Verdict Cache

Content longer than 8,000 characters is evaluated in chunks, with pre-analysis run once over the whole block. The compliant rewrite (the redaction offered in place of a blocked prompt) is produced inline for prompts under 2,000 characters and on demand above that threshold, so verdict latency never waits on a full-document rewrite. A verdict cache, keyed on content, policy state, and engine configuration, serves repeated prompts for six hours without an engine call.

H. Policy Rule Format

The production policy library contains 54 deployed policies across 8 categories: Personally Identifiable Information (PII), Protected Health Information (PHI), General Data Protection Regulation (GDPR), Legal, Code, Confidential, Security, and Finance. The rule field of each deployed policy is a natural-language directive authored by an administrator: an opening directive that names the target class, inline examples of that class, and an exception clause naming what the rule must not catch. An illustrative rule reads: "Block prompts that contain Social Security Numbers. Examples: 123-45-6789, 123456789. Exceptions: masked or placeholder values such as XXX-XX-1234, aggregate statistics, format documentation discussing the pattern xxx-xx-xxxx." Two exception families matter to the benchmark — masked or placeholder values, and fixture values such as test cards, example keys, and documentation addresses — and Section VI treats the format in detail.

I. Admin Autonomy Property

The system turn carries the policies the administrator has enabled for the requesting group, and it instructs the engine to evaluate strictly against those policies and to ignore related prohibitions the model may have internalized during pretraining. We formalize the property this secures as the Admin Autonomy Property (AAP): when an administrator disables policy p for a group, the engine's block rate on prompts crafted to violate p is exactly zero. The property is distinct from accuracy: an engine could be 100% accurate on a fixed policy set and still violate AAP by continuing to block prompts from a disabled policy on the basis of pretraining. In the September 2026 re-validation the engine blocked 0 of 399 prompts crafted to violate disabled policies; Section VII-G reports the protocol.

IV. Benchmark Methodology

We now describe how we constructed the 100,000-prompt benchmark, how we audited its labels, and how we executed and scored it.

A. Corpus Construction

The benchmark comprises $N = 100{,}000$ prompts in six bands (Table I) spanning the 8 categories of Section III-H.

TABLE I
Benchmark bands.

Band	Prompts	Content
Core	36,095	Primary per-facet prompts
Long	3,348	1,000–7,800 characters, single-chunk by design; violation-heavy by construction (buried payloads, split payloads, decoy safes)
Adversarial	6,673	13 obfuscation and evasion techniques
Mixed	9,920	Realistic work prompts pairing a task request with sensitive or masked content
Multilingual	7,548	Non-English prompts in several European languages, across all categories
Benign	36,416	Everyday prompts with no violating content; measures false blocks
Total	100,000	

Each prompt carries a policy_id naming the benchmark test facet it exercises. There are 70 benchmark test facets, and they map many-to-one onto the 54 deployed policies, because several deployed policies are tested at sub-policy granularity. The facet count and the deployed-policy count are distinct and are not conflated elsewhere in this report.

The long band spans 1,000–7,800 characters and is single-chunk by design: every prompt sits below the engine's 8,000-character chunking threshold, so the accuracy numbers measure the single-call path. The band is violation-heavy by construction, with buried payloads, split payloads, and decoy safe content. The adversarial band applies 13 techniques: base64 encoding, dot insertion, fullwidth Unicode, hex encoding, homoglyphs, label/value split, leetspeak, meta-discussion, reversal, social engineering, spacing, spelled-out digits, and vertical layout. The mixed band contains realistic work prompts that pair a task request with sensitive or masked content. The multilingual band contains non-English prompts in several European languages across all categories. The benign band contains 36,416 everyday prompts with no violating content and measures the false-block rate an organization's users would experience.

B. Prompt Types and Label Balance

Each prompt has one of three ground-truth types:

- violation — the prompt contains policy-violating content that the engine should block (block-expected).
- safe — the prompt addresses the same topic domain without violating content (pass-expected).
- edge_case — the prompt contains borderline content (masked values, fixture values, format documentation, questions about policy) that pattern-based systems routinely misclassify but that the engine should pass (pass-expected).

The corpus contains 39,023 block-expected prompts (39.0%) and 60,977 pass-expected prompts (61.0%). By

type there are 38,982 violation prompts (scored), 52,299 safe prompts, and 8,678 edge-case prompts; these type counts are over the 99,959 prompts scored in the primary run, and Section VII accounts for the remaining 41.

We report accuracy (Acc, correct verdicts over total verdicts), the false-positive count (FP: a safe or edge-case prompt incorrectly blocked), and the false-negative count (FN: a violation prompt not blocked), with a Wilson-score 95% confidence interval on accuracy. Because the class balance is designed rather than observed, FP and FN counts are the primary error measures; we report recall (violations blocked) and specificity (pass-expected prompts allowed) as secondary rates.

C. Generation and Validation

Generation is deterministic: every generator runs under a fixed seed, so the corpus regenerates identically. A strict corpus validator then admits the corpus only if it contains no exact duplicates and passes label-balance and length checks, and the resulting manifest is committed with the corpus. Prompt length runs from a minimum of 29 characters through a median of 162 and a 95th percentile of 398 to a maximum of 7,784 characters.

D. Label Audit

Generated labels are only as reliable as the generator's reading of the rule text, so we audited them before scoring. A reader adjudication of 400 randomly sampled misses against the deployed rule texts identified two clusters of label error — a work email address appearing in a routing position, and a public IP address tied to a host — and we corrected 190 labels before scoring. This audit is distinct from the corrected-accuracy protocol of Section VII, which samples residual misses after scoring and credits only those that the rule text decides against the label.

E. Execution and Scoring

We executed the benchmark with a concurrent runner against two compliance engines, each the single-GPU deployment of Section III-B, at 48 concurrent requests. The runner is resumable and checkpoints its results off-box, so an interrupted run resumes from its last checkpoint. The full corpus completes in approximately 2.9 hours at a sustained 9.64 prompts/s. The runner submits prompts to the engine directly, bypassing the enforcement surfaces, so the reported accuracy is the engine's alone.

Scoring compares the verdict with the label: a block-expected prompt is correct when the status is NON-COMPLIANT, and a pass-expected prompt is correct when it is COMPLIANT. A held verdict (Section III-F) is a block. We report raw accuracy first and corrected accuracy second; the corrected figure credits only label errors established by the adjudication protocol of Section VII.

A retest harness with 800 seeded control prompts (400 true blocks and 400 benign) guards the deployed configuration against regression; Section VII reports the retest. The generators, validator, manifest, runner, adjudication protocol and reader outputs, and retest harness are versioned; Section A lists them. The corpus itself is released publicly on Hugging Face under Apache 2.0 (https://huggingface.co/datasets/ShmalexFlow/enterprise-ai-prompt-compliance-100k); the tooling is available to partners on request.

V. Architecture in Depth

Section III describes the compliance engine at the level needed to read the results. This extended description traces the request path end-to-end and states, for each step, why it exists; it then tabulates the escalation triggers and the three completion outcomes, dedicates a paragraph to each enforcement surface, and closes with the locality and caching properties that the efficiency results depend on.

A. Request Path

A single compliance evaluation executes the following ordered steps.

1) Interception. One of the five enforcement surfaces (Section V-D) captures the user-authored prompt before it leaves the organization's boundary and forwards it to the compliance engine. The judgment therefore happens inside the organization's control, before any external AI provider sees the content.
2) Block building. The captured content becomes a single evaluation block. Content longer than 8,000 characters is split into chunks for evaluation. The threshold bounds the per-call input; the benchmark's long band (1,000–7,800 characters) sits entirely below it, so every reported accuracy number measures a single-call verdict.
3) Whole-block pre-analysis. The identifier pre-detector and the canonicalizer (Section III-D) run once over the whole block, before chunking takes effect, so their findings do not depend on where chunk boundaries fall. Both are deterministic: the same content yields the same findings on every evaluation, and neither component can be talked out of a finding.
4) Prompt assembly. The system turn carries the policy prefix — the rule text of all 54 deployed policies, approximately 19,800 tokens — and the user turn carries the content, followed by the pre-analysis section. The prefix is identical across evaluations, which is what allows the inference server to cache it; the pre-analysis section is appended as data, never as instructions, so the deterministic layer informs the judgment without dictating it.
5) Forced-channel call. The inference server is asked for a completion that begins directly in the model's final answer channel, with grammar-enforced structured decoding constraining the output to the verdict schema (status, reason, cited rules, optional rewrite). Because verdict latency scales with output tokens rather than input length (a consequence of the

prefix caching in step 4), the approximately 71-token forced-channel verdict, rather than the approximately 249-token full-reasoning verdict, is the lever that keeps the interactive path fast.

6) Escalation decision. The forced-channel verdict is checked against the triggers of Table II. When a trigger fires, the engine re-judges the same prompt once with full reasoning, and that verdict wins. The triggers are chosen so that the expensive path is spent only where the fast path is least reliable: an allow that coincides with a detected value or a semantic-risk shape, or a block that the model itself attributes to masked content with no value present. About 3% of prompts escalate.
7) Completion classification. Each completion is classified into one of the three outcomes of Table III. Only a usable verdict can allow a prompt; a completion that hits the token ceiling or that the parser cannot use is held and blocked, and only a genuinely unreachable engine follows the organization's configured fail-open or fail-closed setting. The distinction keeps a malformed verdict from ever being mistaken for an outage.
8) Aggregation across chunks. When the block was chunked in step 2, the per-chunk verdicts are combined into a single block-level verdict for the surface. The benchmark does not exercise this step, by construction of the long band.
9) Caching. The verdict is written to a cache keyed on the content, the policy state, and the engine configuration, with a six-hour lifetime. A repeated prompt is served from the cache without an engine call, and keying on policy state means that a rule change never serves a verdict issued under the previous rule text.
10) Rewrite. For a blocked prompt, the compliant rewrite (redaction) is produced inline when the prompt is under 2,000 characters and on demand above that. The gate exists so that verdict latency never waits on a full-document rewrite; the user who wants the rewrite of a long document asks for it, and the user who only needs the verdict is not made to wait.

The benchmark runner of Section IV-E enters this path at step 2, submitting prompts to the engine directly and bypassing the surfaces, so the reported accuracy is the engine's alone.

B. Escalation Triggers

Table II lists the conditions under which a forced-channel verdict is re-judged with full reasoning. Blocked verdicts that do not match the masked-content trigger are never escalated, so the block path stays at forced-channel speed; the escalated minority pays one additional full-reasoning call, typically 4–10 s.

TABLE II
Escalation triggers. A matching verdict is re-judged once with full reasoning, and the second verdict wins.

Forced-Channel Verdict	Trigger and Action
COMPLIANT	Coincides with a pre-analysis value shape (a detected identifier or a canonicalized value): re-judged once
COMPLIANT	Coincides with a semantic-risk signal (schedule, code, formula, money-plan, roster, deal, negotiation, HR-review, litigation, or clinical-context shape): re-judged once
NON-COMPLIANT	Its own reason admits the content is masked, and pre-analysis detected no value: re-judged once
NON-COMPLIANT	Any other case: never escalated

TABLE III
The three completion outcomes.

Outcome	Condition	Disposition
Usable verdict	Completion conforms to the schema within the token ceiling	Allowed or blocked according to its status
Held (fail-closed)	Completion hits the token ceiling, or the parser cannot use it	Blocked with an explicit "compliance incomplete" reason; never allowed
Outage	The engine is genuinely unreachable	The organization's configured fail-open or fail-closed setting

C. Completion Outcomes and Fail-Closed Handling

Table III lists the three outcomes of an engine call. The middle row is the one that matters for a compliance deployment: a verdict the engine could not finish, or could not express in the schema, is a held prompt with an explicit "compliance incomplete" reason, and it is never allowed. The organization's fail-open or fail-closed setting applies to the third row only. We verified the separation on the benchmark (Section VII): after the fix described there, 41 of 41 previously affected prompts block, the 800 seeded controls are unchanged, and 0 of 100,000 prompts fall through to fail-open.

D. Enforcement Surfaces

a) Browser extension.: The Chromium extension targets web-based AI tools. It captures the prompt text in the browser process at the point of submission and holds the page's submit-to-remote path until the engine returns a verdict. A blocked verdict is shown in-page with the cited rules and, where produced, the compliant rewrite; an allowed verdict is transparent to the user.

b) Desktop guard.: The macOS and Windows guards intercept submission to native AI applications that do not traverse a browser, forward the prompt to the engine, and suppress or release the submission according to the verdict.

The guard exists because AI usage on the desktop is not confined to the browser.

c) IDE extension.: The VS Code and JetBrains extensions cover AI-assisted coding interactions, operating at the document and selection level rather than the keystroke level so that IDE responsiveness is preserved. The Code category of the benchmark (credentials, proprietary algorithms, configuration secrets) is the policy material this surface enforces.

d) Infrastructure agent.: A sidecar deployed alongside server workloads (Kubernetes, EC2, Docker) intercepts the outbound path that server-side code takes to call AI APIs. This surface covers pipelines, batch automation, and backend agents where no human types a prompt, and where a leak would otherwise go unseen.

e) Admin dashboard.: The administrator dashboard provides policy management (per-group enablement of each of the 54 deployed policies and their scope), user and group management, activity monitoring, audit search, and forwarding of audit events to the organization's security tooling. It includes a policy-testing playground that issues the same evaluation the surfaces issue, so an administrator can see the verdict, cited rules, and reason a rule produces before enabling it.

E. Locality and Prefix Caching

The compliance engine serves each organization locally, on a single GPU under the organization's control, through the inference server; prompts, verdicts, and the policy prefix stay inside the organization's boundary. The policy prefix is specific to the organization's ruleset and is cached by the inference server, so the approximately 19,800-token prefix is not re-processed on every call and a verdict costs only the prompt's own tokens.

Two efficiency properties follow. First, latency per verdict scales with output tokens rather than input length, which is why the forced channel, at approximately 71 output tokens against approximately 249 for full reasoning, is the dominant lever. Second, the block path is slower than the allow path because a block writes a reason and a rewrite: at 16 concurrent requests the median allowed verdict returns in 1.10 s and the median blocked verdict in 2.69 s. The benchmark ran on two such engines at 48 concurrent requests, sustaining 9.64 prompts/s in aggregate, approximately 4.8 prompts/s per engine; Section VIII reports the full throughput and latency distributions.

VI. Policy-Rule Format in Detail

Section III-H introduces the policy-rule format the compliance engine consumes. Because the rule is the sole administrator-authored contract between an organization's policy and the engine's judgment — and because the corrected-accuracy protocol of Section VII uses the rule text as the arbiter of what the engine should have done — the format's shape is load-bearing. This section specifies the format, walks through four example rules written in the style of the production policy library, describes the two exception families that matter to the benchmark, and describes the administrator authoring workflow.

A. Format

The production policy library contains 54 deployed policies across 8 categories: PII, PHI, GDPR, Legal, Code, Confidential, Security, and Finance. The benchmark exercises these as 70 benchmark test facets, several deployed policies being tested at sub-policy granularity (Section IV-A). Each deployed policy carries a human-readable title, a category, a one-line description, the rule, an example, a risk score, and scope toggles for internal and external evaluation. The engine reads the rule field of each enabled policy as the atomic unit of the policy prefix in the system turn; the other fields appear in the dashboard and the audit log.

The rule field is natural language with three ordered components.

1) Directive opener. An imperative clause that names the target class: "Block prompts that contain Social Security Numbers"; "Block prompts that disclose a specific patient's clinical information with an identifier".
2) Inline examples. A few short canonical examples of the target class after an "Examples:" marker. The examples are literal surface forms, so that the engine can anchor the abstract directive to concrete shapes.
3) Exception clause. An enumerated list of legitimate cases after an "Exceptions:" marker. The exception clause determines the edge-case behaviour: content that pattern systems over-flag — masked and placeholder values, fixture values, format documentation, aggregate statistics, questions about the policy itself — is named explicitly so that the engine permits it.

The format's defining constraint is brevity. A rule is prose, not a domain-specific language; it fits in a single paragraph; it contains no hand-authored regular expressions, no Boolean combinators, and no code. The engine does the work of mapping the prose to the prompt's semantics, and the deterministic pre-analysis of Section III-D supplies the value shapes it would otherwise have to infer.

B. Example Rules

The four rules below are synthetic examples written in the style of production policies in four categories. They are not verbatim from the production policy library but are representative of the production format, and each carries an exception clause from one of the two families described in Section VI-C.

PII — SSN (masked-value exception).

```
Block prompts that contain Social
Security Numbers (SSNs). Examples:
123-45-6789, 123456789, "SSN is
456-78-9012". Exceptions: masked or
placeholder values (XXX-XX-1234,
***-**-6789, [SSN REDACTED]),
aggregate statistics naming SSN as
a field without values, format
documentation of the xxx-xx-xxxx
pattern, questions about SSN
handling policy.
```

Finance — payment cards (fixture-value exception).

```
Block prompts that contain a payment
card number, with or without expiry
date and security code. Examples: a
15- or 16-digit card number, spaced,
dashed, or unbroken, alone or with
"exp 09/28, CVV 123". Exceptions:
masked numbers (**** **** **** 1234),
industry test cards (4111 1111 1111
1111, 5555 5555 5555 4444, 3782
822463 10005), format documentation
of card-number structure, questions
about card-handling policy.
```

Code — credentials (fixture-value exception).

```
Block prompts containing live
credentials, API keys, or secrets in
code, configuration, or prose.
Examples: "AWS_ACCESS_KEY_ID=AKIA"
followed by a full key; "export
SERVICE_API_KEY=" followed by a full
key; a personal access token pasted
into a script. Exceptions:
placeholder keys ("sk-XXXX",
"your-api-key-here", "<REDACTED>"),
vendor documentation example keys
(AKIAIOSFODNN7EXAMPLE), environment
variable names with no value,
discussions of key rotation.
```

Security — network topology (documentation-address exception).

```
Block prompts that disclose internal
network topology: hostnames paired
with their addresses, subnet maps,
firewall rules, or VPN endpoints.
Examples: "prod-db-01 is at
10.4.12.7"; a pasted routing table;
"our VPN concentrator is
vpn.corp.internal on port 4443".
Exceptions: documentation address
ranges (192.0.2.0/24, 198.51.100.0/24,
203.0.113.0/24) and example.com
hostnames, masked addresses
(10.x.x.x, 192.168.0.0/16 named as a
range), textbook subnetting
exercises, questions about network
security policy.
```

Two properties of these examples are worth calling out. First, every rule names both a target class and a list of permitted uses; the engine is told what to block and what to pass. Second, no rule contains a regular expression, a keyword list, or a decision tree. The surface shapes in "Examples:" are there for the model to generalize from, not for a pattern matcher to match.

C. The Two Exception Families the Benchmark Depends On

Across the 54 deployed policies, two exception families matter to the benchmark, and both are enforced by rule text rather than by code.

Masked and placeholder values. A value whose digits or characters are partly or wholly replaced (XXX-XX-1234, **** 1234, [REDACTED], <your-api-key>) is not the protected value, and the rule says so explicitly. The engine treats this family at three points: the identifier pre-detector excludes placeholders, so no value shape is asserted for them; the exception clause tells the model to pass them; and a NON-COMPLIANT verdict whose own reason admits that the content is masked, with no value detected, is re-judged once with full reasoning (Section III-E). The family remains the largest source of over-blocks in the error analysis of Section VII, and the corrected-accuracy protocol counts a masked value blocked despite an explicit rule exception as an engine error, not a dataset error, so those over-blocks are never credited back.

Fixture values. Industry test card numbers, vendor documentation example keys, and reserved documentation address ranges are published precisely so that they can appear in code, tests, and manuals without disclosing anything. The rule names them as exceptions so that a developer pasting a test fixture or a network engineer quoting a documentation range is not blocked.

The deployed configuration carries these two exception families in its rule text. On the same 100,000 prompts under the same scoring, the forced-channel configuration with escalation and pre-analysis but without these clauses scores 95.49% raw (96.16% corrected); the deployed configuration with them scores 96.53% raw (96.78% corrected). In a same-prompt comparison of the two, the clauses fixed 1,648 verdicts and changed 800 the other way, a net gain of 848 (Section VII).

D. Authoring Workflow

Administrators author rules from the dashboard without touching code. The workflow is:

1) The administrator creates a policy in the dashboard, selects a category, and enters a title and one-line description.
2) The administrator writes the rule in natural language, following the three-component structure (directive, examples, exceptions), and names the masked, placeholder, and fixture forms the rule must not catch.
3) The administrator sets the risk score (used for audit ranking, not for enforcement), sets internal and external scope, and selects the target groups.
4) The administrator tests the rule in the dashboard playground against representative prompts. The playground issues the same evaluation the enforcement surfaces issue and shows the verdict, cited rules, and reason.
5) The administrator saves the policy. The saved rule text becomes part of the policy prefix for subsequent evaluations against a group in scope, and, because the verdict cache is keyed on policy state, no verdict issued under the previous rule text is served for it.

No code deployment is required; no rule parser is involved; no release train gates a rule change. This matters for compliance operations because the people who own policy — legal, compliance, and security administration — are not typically the people who own the code release process.

E. Why the Format Is Minimal

Three measured facts justify the brevity of the format. First, the rule text is the reference against which both the label audit (Section IV-D) and the corrected-accuracy protocol (Section VII) are adjudicated; in the latter, eight independent readers decide each sampled miss by reading the deployed rule and the prompt, and a rule that a reader can hold in one paragraph is a rule that can be adjudicated consistently. Second, the exception clause is a measurable lever, as the configuration comparison above shows, and it is one an administrator can pull from the dashboard without a release. Third, the remaining error mass is judgment-shaped — semantic violations with no lexical shape, and masked-value over-blocks — and does not yield to further rule-text engineering at forced-channel speed (Section XI-G); longer rules would add tokens to the policy prefix without moving those errors. We collected every accuracy number reported in Section VII under this format.

VII. Results

We report the deployed configuration of the compliance engine on the full benchmark of $N = 100{,}000$ prompts across the eight categories and six bands of Section IV-A. The headline is 96.53% raw accuracy and 96.78% corrected accuracy (95% confidence interval (CI) 96.65–96.95), sustained at 9.64 prompts/s on two engines with a median latency of 1.10 s for allowed prompts at 16 concurrent requests. Sections VII-A–VII-E report accuracy; Section VII-F documents the fail-closed validation; Section VII-G validates the Admin Autonomy Property; Section VII-H reports throughput and latency.

A. Overall Accuracy

Of the 100,000 prompts, 99,959 received a scorable verdict on the primary run, and 96,491 of those verdicts were correct, for a raw accuracy of Acc = 96.53%. The remaining 41 prompts returned no verdict on the primary run; they are the subject of Section VII-F, and after the fix described there all 41 block, so every one of the 100,000 prompts now receives a verdict.

The 3,468 misses divide into FN = 1,391 false negatives (violation prompts allowed) and FP = 2,077 false positives (pass-expected prompts blocked). Recall, the share of violation prompts blocked, is Rec = 96.43%; specificity, the share of pass-expected prompts allowed, is Spec = 96.59%. On the benign band of 36,416 everyday work prompts the pass rate is $R_{\text{benign}} = 99.89\%$: 40 false blocks in 36,416.

a) Corrected accuracy.: A miss is a disagreement between the engine and a corpus label, and corpus labels are not infallible. We therefore adjudicated the misses against the deployed rule text, the only authority on what the engine should have done, and report a corrected accuracy alongside the raw one. The adjudication was performed on the 3,658 misses of the run before label correction. Two mechanical classes are decided from the rule text alone: D1, the corporate-email carve-out, where the rule text exempts the address the label counts as a violation (13 rows, a dataset error), and M1, a masked or placeholder value blocked despite an explicit rule exception (471 rows, an engine error). From the 3,174 residual misses we sampled 400 at random (200 FN and 200 FP; 21 removed as mechanically decided, 379 read), and eight independent readers judged each sampled prompt against the rule text. Where the rule verdict differs from the label the miss is a dataset error and is credited to the engine; where the two agree it is an engine error and is not. The readers attributed 18.0% of sampled FN [13.3, 23.9] and 8.9% of sampled FP [5.6, 14.0] to the dataset (Wilson 95% intervals). Combining the mechanical classes with those shares gives a corrected accuracy of $\text{Acc}_{\text{corr}} = 96.78\%$ (95% CI 96.65–96.95). Eighteen sampled calls were arguable either way; crediting them would give 96.92%, and we do not. The reading also exposed two label clusters totalling 190 rows, which we then corrected in the corpus, leaving the 3,468 misses (1,391 FN, 2,077 FP) reported above, of which roughly 240 are estimated to remain label-side; the corrected figure is unchanged by the corrections because the sample had already credited those clusters. The full protocol and the reader outputs are versioned with the corpus, and the protocol is given in full in Section IX.

B. Accuracy by Band

Table IV reports accuracy on each of the six bands. The benign band, the closest proxy for everyday traffic, allows 99.89% of its prompts. The multilingual and core bands sit near the corpus mean. The three bands built to be hard—mixed (a realistic task request paired with sensitive or masked content), long (1,000–7,800 characters, single-chunk by design), and adversarial (thirteen obfuscation techniques)—are where the misses concentrate; the explanation is in Section VII-E and Section IX.

C. Per-Category Results

Table V reports accuracy, false negatives, and false positives for each of the eight categories. Every category except one is above 96.4%, and four—PHI, Security, Finance, and Legal—are above 96.7%. Confidential is the category floor at 93.01%, and its 590 false negatives are the largest false-negative count of any category by a wide margin: roadmaps, board minutes, incident narratives, and contract terms are violations of kind rather than of token, with no identifier for pre-analysis to detect. PII carries the most false positives (469), concentrated on masked,

TABLE IV
Accuracy by band, deployed configuration. Band counts include the 41 prompts of Section VII-F; accuracy is over scored verdicts.

Band	Prompts	Acc. (%)
Benign	36,416	99.89
Multilingual	7,548	96.03
Core	36,095	95.89
Mixed	9,920	92.25
Long	3,348	91.76
Adversarial	6,673	90.99
All bands	100,000	96.53

TABLE V
Per-category results, deployed configuration. FN = violation prompt allowed; FP = pass-expected prompt blocked.

Category	Prompts	Acc. (%)	FN	FP
PHI	15,031	97.94	77	232
Security	11,322	97.77	66	187
Finance	9,256	97.23	81	175
Legal	10,978	96.73	102	257
Code	12,153	96.64	194	214
GDPR	11,909	96.60	153	252
PII	16,701	96.43	128	469
Confidential	12,609	93.01	590	291
All	99,959	96.53	1,391	2,077

placeholder, and fixture values that have the shape of live identifiers and are not (Section IX).

D. Per-Type Results

The class balance is designed rather than observed: 39,023 prompts (39.0%) are block-expected and 60,977 (61.0%) are pass-expected. Table VI disaggregates by prompt type. Violation prompts (39,023 labelled; 38,982 scored, the difference being the 41 prompts of Section VII-F) are blocked at 96.43%. Safe prompts are allowed at 96.99%. Edge-case prompts, the borderline content of the prompt taxonomy (Section IV-B), are allowed at 94.18%; they are the type on which the forced final channel over-blocks most, and Section IX sorts those over-blocks by mechanism.

E. Accuracy by Length

Table VII reports accuracy by prompt length in characters. Accuracy declines with length, from 97.58% under 200 characters to 90.08% between 4,000 and 8,000, and we read the decline for what it is. The long buckets are small (386, 1,453, 1,588, and 262 prompts above 500 characters), and the buckets above 1,000 characters correspond to the long band, which is violation-heavy by construction: buried payloads, split payloads, and decoy safe prompts were written to be hard. Those buckets therefore oversample the hard block path rather than sample long documents at large. The benign long-form share allows at approximately 99%. Two further facts bound the effect. Every prompt in the corpus is judged as a single block, because the chunking threshold is 8,000 characters and the long band tops out at 7,800, so the decline is not a chunking artefact; and latency does not grow with length, because the policy prefix is cached and each verdict pays only for its own tokens (Section VII-H).

TABLE VI
Results by prompt type, deployed configuration.

Type	Prompts	Expected	Correct (%)
violation	38,982	block	96.43
safe	52,299	allow	96.99
edge_case	8,678	allow	94.18

TABLE VII
Accuracy by prompt length, deployed configuration.

Length (chars)	Prompts	Acc. (%)
<200	69,555	97.58
200–500	26,715	94.43
500–1,000	386	93.78
1,000–2,000	1,453	93.67
2,000–4,000	1,588	90.30
4,000–8,000	262	90.08

F. Fail-Closed Validation

The engine is designed to fail closed: a verdict that hits the token ceiling, or a completion the parser cannot use, is held—blocked with an explicit "compliance incomplete" reason—and is never allowed. Only a genuinely unreachable engine follows the organization's configured fail-open or fail-closed setting (Section III). The 100,000-prompt run tested that design and found one gap. Forty-one prompts, all instances of one proprietary-algorithm template carrying a patent and privilege line, returned "compliance unavailable" and were allowed.

We established the root cause by capturing the engine's raw completion. The inference server silently ignored the legacy schema-constraint field, so the grammar cap on the cited-rule list was not enforced; at temperature 0 the model looped on a single rule reference, never closed the array, and the parser rejected the completion. The unparseable path was treated as an outage and inherited the fail-open setting.

The fix has two parts. The engine now uses the current structured-output field, whose schema enforcement we verified by replay (190 tokens, valid JSON, cap honoured), and any unparseable completion now follows the fail-closed rule. Verification: all 41 prompts now block (41/41); the 800 seeded control prompts show no regression (399/400 true blocks held, 400/400 benign prompts allowed, 0 unavailable); and 0 of the 100,000 corpus prompts fall through to fail-open.

G. Admin Autonomy Property Validation

The Admin Autonomy Property (Section III-I) requires that enforcement follow the administrator's configuration

and nothing else. For the September 2026 re-validation we disabled the card-number policy for the test group, waited out the policy cache, and ran the 399 core prompts that violate it, with the policy enabled and again with it disabled. The property has two levels, and the engine meets one of them. At the attribution level it holds: with the policy disabled, 0 of 399 verdicts cited it. At the block-rate level it does not: of the 318 prompts that, with the policy enabled, had been blocked citing that policy alone, all 318 remained blocked with it disabled, attributed instead to neighbouring enabled policies (card verification data 220, credentials 78, bank accounts 22). For 113 of them the neighbouring citation is defensible because the prompt also carries an expiry date, a verification code, a credential, or an account number; for the other 205 the prompt contains a card number and nothing else, and the verdict reached for the nearest enabled rule. The forced channel, which has no room to reason about scope, over-attributes to adjacent rules even though the system prompt places the autonomy directive first. We then tested the obvious remediation: a scope sentence on each of the three neighbouring rules stating that a card number on its own is governed solely by the card-number policy. With that text deployed and seeded, the same experiment blocked 194 of 195 bare-card-number prompts under the same neighbouring rules; no verdict cited the disabled policy in either run. Rule text does not move the forced channel on this question, just as it does not move it on masked values (Section IX). A final variant removes any definitional explanation: with the card-number policy and all three neighbouring policies disabled together, 194 of 195 bare-card-number prompts were still blocked, attributed to a further ring of rules (transaction and payment credentials 179, authentication secrets 33, wire-transfer routing 3), with stated reasons of the form "contains a real credit card number". No enabled rule's definition covers those prompts; the forced channel enforces the card number on its own and attaches the block to the nearest enabled rule. Stating the switched-off classes explicitly in the prompt did not change that either (188 of 194).

The remedy that holds is deterministic. After the engine's citations are resolved to policies, a guard checks whether the text carries the value shape of a policy the administrator has switched off, whether every cited policy is one whose own value shape the guard recognises, and whether none of those shapes is present; only then is the block dropped and the re-attribution logged. With the guard deployed, the same experiment allows 191 of 197 bare-card-number prompts with the card-number policy off, keeps 94 of 102 prompts that also carry a verification code, expiry, credential, or account number blocked under those rules, cites the disabled policy in 0 verdicts, and returns every probed prompt to a block once the policy is restored. The six bare-card-number prompts still held were filed under rules outside the guard's table (government identifiers, payroll, settlement agreements), which the guard never overrides by design. The property therefore holds mechanically for identifier-class policies; for semantic policies, whose content has no value shape, it remains an open item on the fine-tuning track (Section XI-G).

TABLE VIII
Throughput and latency, deployed configuration, two engines. Latency is end-to-end per verdict.

Measure	Value
Throughput, two engines, 48 concurrent	9.64 prompts/s
Throughput per engine	≈4.8 prompts/s
Wall time, 100,000 prompts	≈2.9 h
Median latency, allowed, 16 concurrent	1.10 s
Median latency, blocked, 16 concurrent	2.69 s
p90 latency, all verdicts, 16 concurrent	4.88 s
Median latency, allowed, 48 concurrent	2.18 s
Median latency, blocked, 48 concurrent	5.07 s
Output tokens per verdict, forced channel	≈71
Output tokens per verdict, full reasoning	≈249
Throughput, full-reasoning configuration	4.08 prompts/s

H. Throughput and Latency

Speed is a design property of the engine, and we report it with the same weight as accuracy (Table VIII). Sustained throughput on the full run is 9.64 prompts/s on two engines at 48 concurrent requests, ≈4.8 prompts/s per engine; the 100,000-prompt corpus completes in approximately 2.9 hours. At 16 concurrent requests the median latency is 1.10 s for an allowed verdict and 2.69 s for a blocked one, with an overall p90 of 4.88 s; at 48 concurrent the medians are 2.18 s and 5.07 s.

Two mechanisms produce these figures. The forced final channel generates ≈71 output tokens per verdict where full chain-of-thought reasoning generates ≈249, a 3.5× reduction; the same engine run with full reasoning sustains 4.08 prompts/s. Blocked verdicts are slower than allowed ones because they write a reason and, for prompts under 2,000 characters, an inline compliant rewrite; they are otherwise never escalated, so the block path stays at forced-channel speed. Latency scales with the tokens the engine writes, not with the length of the prompt it reads, because the policy prefix is cached once by the inference server. Section VIII gives the full efficiency analysis and Section X the throughput of every configuration measured.

VIII. Efficiency Analysis

Section VII-H reported the headline speed figures: 9.64 prompts/s sustained on two engines, a 1.10 s median for allowed verdicts at 16 concurrent requests, and ≈71 output tokens per verdict. This section explains where those figures come from and what governs them. The short version is that per-verdict latency in the deployed engine is set by the number of tokens the engine writes, not by the number it reads, and the deployed configuration is built to write as few as a defensible verdict allows.

TABLE IX
Latency by concurrency and verdict, deployed configuration, two engines. Values are medians except the last column.

Concurrent	Allowed (s)	Blocked (s)	p90, All (s)
16	1.10	2.69	4.88
48	2.18	5.07	—

A. Latency Follows Output Tokens, Not Input Length

Every evaluation places the full administrator-authored ruleset in the system turn: 54 deployed policies as natural-language rules with inline examples and exception clauses, about 19,800 tokens. Read naively, that prefix would dominate every request. It does not, because the inference server prefix-caches it: the policy prefix is processed once and reused, and every subsequent verdict pays only for the prompt's own tokens. Those are few. The corpus median is 162 characters and the 95th percentile 398 (Section IV-A), so the input-side cost of a verdict is small, and latency does not scale with input length. It scales with generation, and generation is governed by the shape of the verdict.

B. The Forced Channel's Token Economy

The verdict is produced on a forced final channel: the model is prompted directly into its answer channel with no room for chain-of-thought, and the output is schema-constrained (status, reason, cited rules, optional rewrite) by grammar-enforced structured decoding (Section III). The effect on token count is large. A forced-channel verdict is ≈71 output tokens; the same engine with full reasoning writes ≈249, a 3.5× difference, most of it deliberation that never reaches the user. The forced channel alone sustains 10.73 prompts/s where full reasoning sustains 4.08; escalation and pre-analysis, which buy back most of the accuracy the forced channel gives up, bring the deployed configuration to 9.64 (Section X).

C. Block Path Versus Allow Path

The two verdict outcomes have different token budgets and therefore different latencies. An allowed verdict is short: a status, a brief reason, and no rewrite. A blocked verdict must also cite the rules it relies on and, for prompts under 2,000 characters, produce the compliant rewrite (the redaction) inline, so that the enforcement surface can offer it to the user at once. Writing that rewrite is where the block path spends its extra time. Table IX reports the medians. At 16 concurrent requests an allowed verdict returns in 1.10 s and a blocked one in 2.69 s, with an overall p90 of 4.88 s; at 48 concurrent the medians rise to 2.18 s and 5.07 s as requests queue for the same two engines. Blocked verdicts are never escalated except in the single masked-value case described below, so the block path runs at forced-channel speed.

D. The Deferred-Rewrite Gate

The inline rewrite is bounded by a length gate. For prompts under 2,000 characters the compliant rewrite is produced inline with the verdict; this is the interactive case and the case Table IX measures. Above 2,000 characters the verdict returns without a rewrite and the rewrite is produced on demand, so a verdict on a long document never waits for the engine to rewrite that document. Content over 8,000 characters is chunked, with pre-analysis run over the whole block before any chunk is judged. Together the two thresholds keep verdict latency a function of the verdict, not of the document.

E. Escalation Cost

The hybrid escalation path is the one place the deployed engine spends full-reasoning tokens. A forced-channel COMPLIANT verdict that coincides with a pre-analysis value shape or a semantic-risk signal—a schedule, code, formula, money plan, roster, deal, negotiation, HR review, litigation, or clinical-context shape—is re-judged once with full reasoning, and that verdict wins. A NON-COMPLIANT verdict whose own reason admits the content is masked, with no detected value, is re-judged the same way. About 3% of prompts escalate. Each escalation is one additional full-reasoning call, typically 4–10 s, which is why the escalating configuration runs at 9.61 prompts/s where the forced channel alone runs at 10.73. The design confines that cost to the prompts where the forced channel's own output signals doubt, and, apart from the masked-value case, never to blocked verdicts, so the ordinary user experience remains the forced-channel latency of Table IX. A verdict cache, keyed on content, policy state, and engine configuration and held for six hours, serves repeated prompts without any engine call.

F. Throughput per Engine and Deployment Sizing

An engine is one instance of the model, served locally on a single GPU under the organization's control. On the full 100,000-prompt run two engines sustained 9.64 prompts/s at 48 concurrent requests, ≈4.8 prompts/s per engine, and completed the corpus in approximately 2.9 hours. The two-engine figure is two engines working independently on the same policy prefix, so, as a first approximation, capacity planning proceeds per engine: an organization sizes its pool to its expected peak prompts per second at roughly 4.8 per engine. Table X places the deployed configuration against the alternatives measured on the same corpus.

G. Control Retest

The retest harness runs 800 seeded control prompts—400 true violations and 400 benign prompts—against the deployed configuration. On the current configuration 399 of 400 true blocks held, 400 of 400 benign prompts were allowed, 0 returned unavailable, and the median latency was 1.61 s. That median sits between the allowed and blocked medians at 16 concurrent, as expected for a set that is half blocks and half allows, and the same harness verified the fail-closed fix of Section VII-F.

TABLE X
Throughput by configuration on the full corpus, two engines. Output tokens are per verdict, approximate.

Configuration	Output Tokens	Prompts/s
Full chain-of-thought reasoning	≈249	4.08
Forced final channel alone	≈71	10.73
Forced channel + escalation and pre-analysis	≈71, plus one reasoning call on ≈3%	9.61
Deployed (+ masked-record and fixture exception clauses)	as above	9.64

H. The Full-Reasoning Configuration

For comparison we ran the same engine with full chain-of-thought reasoning on the same corpus. It sustains 4.08 prompts/s and writes ≈249 output tokens per verdict; its accuracy is 96.39% raw and 96.92% corrected. The deployed hybrid reaches 96.78% corrected, within 0.14 points, at 2.4× the throughput. Section X reports the full configuration comparison, including what the forced channel alone gives up and what escalation and pre-analysis recover.

IX. Error Analysis

Section VII-A reported 3,468 misses on the scored corpus after label correction: 1,391 false negatives and 2,077 false positives. The adjudication and the pool census in this section were performed on the 3,658 misses of the run before the 190 label corrections, and we present them that way. This section does three things with them. It gives in full the adjudication protocol behind the corrected accuracy; it sorts the misses into mechanism pools and reports how much of each pool is the engine's rather than the corpus's; and it says what the pools imply about the lever that remains.

A. Adjudication Protocol

A miss is a disagreement between the engine and a corpus label. The label records a generator's intent; the deployed rule text is what the engine is asked to enforce; and the two can differ. The protocol therefore judges every adjudicated miss against the rule text, and credits the engine only where the rule text sides with it.

a) Mechanical classes.: The audit read 400 of the 3,658 pre-correction misses, which divide into 13 D1 rows, 471 M1 rows, and 3,174 residual misses. Two classes can be decided from the rule text without a reader. D1 is the corporate-email carve-out: the rule text exempts the address that the label counts as a violation. D1 holds 13 rows and is a dataset error. M1 is a masked or placeholder value blocked despite an explicit rule exception: the rule text exempts the value, and the engine blocked it anyway. M1 holds 471 rows and is an engine error.

b) Reader sample.: The residual 3,174 misses were sampled at random: 200 false negatives and 200 false positives. Twenty-one of the 400 turned out to be mechanically decided and were removed, leaving 379. Eight independent readers read each of the 379 against the deployed rule text and recorded a rule verdict. The standard was fixed in advance. If the rule verdict differs from the corpus label, the miss is a dataset error and is credited to the engine; if the rule verdict agrees with the label, the miss is an engine error and is not credited. A reader could also mark a call arguable.

c) Shares and corrected accuracy.: The readers attributed 18.0% of sampled false negatives to the dataset (Wilson 95% interval [13.3, 23.9]) and 8.9% of sampled false positives ([5.6, 14.0]). Combining the two mechanical classes with those shares yields the corrected accuracy of 96.78% (95% CI 96.65–96.95) reported in Section VII-A. Eighteen calls were marked arguable; crediting them to the engine would raise the figure to 96.92%, and we report the figure without them.

d) Label corrections.: The same reading exposed two systematic label clusters in the corpus: a work email address in a routing position, and a public IP address tied to a host. We then corrected the 190 rows in those clusters, leaving the 3,468 misses (1,391 false negatives, 2,077 false positives) that Section VII reports, of which roughly 240 are estimated to remain label-side. The corrected figure of 96.78% is unchanged by the corrections, because the sample had already credited those clusters. The protocol, the reader outputs, and the corrections are versioned with the corpus (Appendix A).

B. Error Mass by Mechanism

Table XI is the census of the 3,658 pre-correction misses, sorted into mechanism pools. For each pool it gives the number of misses and the share of the pool's sampled rows that the readers attributed to the engine rather than to the corpus label. The pools are ordered by size. The first six carry most of the mass and engine shares between 81% and 95%; the first two are the judgment-shaped core to which Section IX-D returns.

C. Mechanisms

a) Semantic violations with no lexical shape (780; 87% engine).: A product roadmap, a proprietary algorithm described in prose, an incident narrative, a set of board minutes, or a contract clause violates a Confidential, Legal, or Code rule without containing a single token that pre-analysis can detect. The forced channel must recognize the kind of document from prose alone, with no chain-of-thought in which to weigh it, and it under-calls. The semantic-risk signals that trigger escalation (schedule, code, formula, money plan, roster, deal, negotiation, HR review, litigation) reach the shapes they name; what remains is the long tail of documents that match none of them. This pool is the reason Confidential is the category floor at 93.01% with 590 false negatives (Table V).

TABLE XI
Misses by mechanism pool, deployed configuration: the census of the 3,658 misses before label correction. Engine share is the fraction of the pool's sampled rows that readers attributed to the engine rather than to the corpus label.

Pool	Misses	Engine (%)
Missed semantic violations with no lexical shape (roadmaps, algorithms, incidents, board minutes, contracts)	780	87
Over-blocks on masked or placeholder values	542	95
Over-blocks in benign context	461	90
Over-blocks on questions about policy	396	92
Over-blocks on fixture values	324	88
Over-blocks on decoded benign obfuscation	312	81
Missed clinical or biometric named-person context	248	76
Missed obfuscated values	192	80
Missed identifier context	176	33
Over-blocks on long benign documents	84	44
Missed multilingual semantic content	82	71
Missed buried needles	40	62
Empty document shells	21	—
All misses before label correction	3,658	

b) Masked-value over-blocks (542; 95% engine).: The prompt contains a value that is visibly masked or a placeholder (a redacted or templated field), and the rule text carries an explicit exception for exactly that. The forced verdict nonetheless asserts that the masked value is real, and blocks. The hybrid catches the sub-case in which the verdict's own reason admits the value is masked and re-judges it; the pool that remains is the one in which the verdict asserts the value is real, so nothing in its output signals doubt and nothing triggers a re-judge. The M1 mechanical class of Section IX-A is the sharpest form of this mechanism. It is the largest false-positive pool and, at 95%, the most engine-attributable pool in the table.

c) Benign context (461; 90% engine).: A prompt whose subject belongs to a governed category but whose content discloses nothing the rule names: an ordinary work request written in the vocabulary of a sensitive domain. The forced channel over-blocks on the vocabulary.

d) Policy meta-questions (396; 92% engine).: A prompt that asks about a policy—what the organization's rule on a class of data is, whether a kind of content is permitted—names the governed class without disclosing anything in it. The forced verdict keys on the subject rather than on the act, and blocks.

e) Fixture values (324; 88% engine).: Documentation and test fixtures use values that have the shape of live data and are not: the sample numbers and example records that appear in manuals and test suites. The deployed configuration adds fixture exception clauses to the rule text, which is part of the gain reported in Section X; the 324 misses here are the fixtures the forced channel still treats as live.

f) Decoded benign obfuscation (312; 81% engine).: The canonicalizer reverses encodings mechanically and appends what it finds to the evaluation input as data. When the decoded content is itself benign, the forced channel over-blocks: it reads the presence of an encoding as evidence of evasion rather than judging the decoded content on its merits. The pre-analysis that closes obfuscated violations in the adversarial band is thus also the source of a false-positive pool on their benign counterparts.

g) Clinical and biometric context (248; 76% engine).: A named person paired with a clinical or biometric fact is protected health information whether or not a record number or any other identifier shape is present. The identifier pre-detector has nothing to detect, the forced channel has no room to reason about the pairing, and the clinical-context shape among the escalation signals does not cover every form the pairing takes.

h) Obfuscated values (192; 80% engine).: The canonicalizer reverses obfuscation by family: unicode and homoglyph folding, separator stripping, vertical layouts, base64 and hex decoding, leetspeak, spelled-out digits, announced reversals, and split halves. The 192 misses are obfuscated values that survive canonicalization and that the forced channel does not decode on its own. Adding canonicalizer families is the direct lever, and Section X-E measures it at +6 of 192.

i) Smaller pools.: The remaining pools are small and carry lower engine shares. Missed multilingual semantic content (82; 71%) is the non-English counterpart of the first pool. Missed identifier context (176; 33%) and over-blocks on long benign documents (84; 44%) are majority dataset-side. The 190 corrected rows fall almost entirely in that pool (127) and in the network-identifier over-block pool (63), which is where the two label clusters live. Missed buried needles (40; 62%) are the long band's single-mention payloads. Twenty-one misses are empty document shells—the frame of a document with no content—and we report them without attribution.

D. What Remains

Two facts follow from Table XI. First, the error mass is the engine's. The six largest pools carry engine shares between 81% and 95%, which is why adjudication moves the headline by only a quarter of a point (96.53% to 96.78%): the misses that remain are ours to fix, not the corpus's. Second, the two largest pools are judgment-shaped. A semantic violation with no lexical shape and a masked value asserted to be real are both errors of reading, not of detection. Together they are about 1.3 percentage points, and they do not yield to prompt or rule-text engineering at forced-channel speed; the deployed configuration's exception clauses are the most recent such lever, and Section X reports what they returned. A full-reasoning re-judge of masked-record blocks fixes about half of the masked over-blocks, but at 8–10 s per block, and we did not adopt it because interactive latency is a product

TABLE XII
Configuration comparison on the full corpus. Corrected accuracy applies the adjudication of Section IX-A; p/s is sustained throughput in prompts per second on two engines.

Configuration	Raw (%)	Corr. (%)	p/s
Full chain-of-thought reasoning	96.39	96.92	4.08
Forced final channel alone	94.78	95.02	10.73
Forced channel + escalation and pre-analysis	95.49	96.16	9.61
Deployed: + masked-record and fixture exception clauses	96.53	96.78	9.64

requirement (Section X-D). Mechanical levers are likewise nearly spent: added canonicalizer families measure at +6 of the 192 missed obfuscated values.

The lever that remains is the forced channel itself. The engine already produces two kinds of verdict that the forced channel never sees at training time: the adjudicated verdicts of this section and the full-reasoning teacher verdicts of the escalation path. Supervised fine-tuning of the forced channel on those verdicts, with the benign band held as the guardrail against trading false negatives for false blocks, is the path forward. With every engine-attributable defect in Table XI fixed, the ceiling on this corpus is approximately 99.6%.

X. Configuration Comparison

The deployed configuration is one point in a small design space: how much the model is allowed to reason, what deterministic pre-analysis it is given, when a verdict is re-judged, and what the rule text exempts. We measured four configurations on the same 100,000-prompt corpus under the same scoring and the same adjudication (Table XII), so that each difference is attributable to the configuration alone.

A. Four Configurations

Full chain-of-thought reasoning lets the model deliberate before every verdict. It is the accuracy reference: 96.39% raw and 96.92% corrected, at 4.08 prompts/s and ≈249 output tokens per verdict. Forced final channel alone removes the deliberation and constrains the output to the verdict schema: 94.78% raw and 95.02% corrected, at 10.73 prompts/s. Forced channel with escalation and pre-analysis adds the identifier pre-detector, the canonicalizer, and the hybrid re-judge of Section III: 95.49% raw and 96.16% corrected, at 9.61 prompts/s. The deployed configuration adds masked-record and fixture exception clauses to the rule text: 96.53% raw and 96.78% corrected, at 9.64 prompts/s.

The two scorings order the top two configurations differently. On raw accuracy the deployed configuration is ahead of full reasoning (96.53% against 96.39%); on corrected accuracy full reasoning is ahead by 0.14 points. We take the corrected ordering as the fair one, since it is the one judged against the rule text, and state the result as: the hybrid recovers to within 0.14 corrected points of full reasoning at 2.4× the throughput.

B. Same-Prompt Comparison of the Last Two

Aggregate accuracy can hide churn, so we compared the last two configurations prompt by prompt. Adding the exception clauses fixed 1,648 verdicts and changed 800 the other way, a net gain of 848. The 800 regressions are the reminder that a rule-text change moves verdicts in both directions, and that only the net, measured on the same prompts, is evidence. The net is positive by a wide margin and the change costs nothing in throughput (9.61 to 9.64 prompts/s), which is why it is deployed.

C. What the Forced Channel Loses, and What Recovers It

Removing deliberation costs 1.61 raw points and 1.90 corrected points against full reasoning (96.39% to 94.78%; 96.92% to 95.02%). The loss falls where the verdict depends on reading rather than on detection: a masked value the model must recognize as masked, a document whose sensitivity lies in its kind rather than in any token, an obfuscated value it must decode unaided. In exchange the forced channel writes ≈71 output tokens rather than ≈249 and runs 2.6× faster.

Escalation and pre-analysis recover 0.71 raw points and 1.14 corrected points of that loss (94.78% to 95.49%; 95.02% to 96.16%) for about a tenth of the forced channel's throughput (10.73 to 9.61 prompts/s). Pre-analysis supplies as data what the forced channel cannot derive inside the answer channel: detected identifiers, and the canonical form of obfuscated text. Escalation sends the roughly 3% of verdicts in which the forced output coincides with a value shape or a semantic-risk signal to one full-reasoning call, and takes that call's verdict. The exception clauses recover a further 1.04 raw and 0.62 corrected points (95.49% to 96.53%; 96.16% to 96.78%) at no throughput cost. What remains between the deployed configuration and full reasoning is 0.14 corrected points; what remains between either and the ceiling is the judgment-shaped mass of Section IX-D.

D. Masked-Record Re-Judge: A Negative Result

The largest false-positive pool is the masked-value over-block (Section IX-C), and the direct remedy is a full-reasoning re-judge of masked-record blocks. We measured it. The re-judge fixes about half of the masked over-blocks. It also costs 8–10 s per block, because it places a full-reasoning call on the block path, which the deployed design otherwise keeps at forced-channel speed. We did not adopt it. Interactive latency is a product requirement: a user whose prompt is blocked is waiting for the verdict and the rewrite, and an 8–10 s block is outside the interactive budget. The masked over-blocks stay in the error mass and go to the fine-tuning path of Section IX-D instead.

E. Canonicalizer Families

The mechanical lever on the false-negative side is the canonicalizer. Adding families to it—extending the set of obfuscations reversed before the model sees the text—is cheap and deterministic, and we measured its return on the 192 missed obfuscated values of Table XI: +6. The families already deployed (unicode and homoglyph folding, separator stripping, vertical layouts, base64 and hex decoding, leetspeak, spelled-out digits, announced reversals, split halves) are the ones with a clean mechanical inverse; what survives them is mostly obfuscation the model has to read, and it belongs with the judgment-shaped mass rather than with the mechanical levers.

a) Summary.: Across the four configurations the deployed point is the accuracy–throughput trade we chose: within 0.14 corrected points of full reasoning at 2.4× its throughput. The two experiments not adopted—the masked-record re-judge and further canonicalizer families—bound what remains to be had from engineering around the forced channel, and point the remaining error mass at training it.

XI. Discussion

A. Reading the Headline Against the Alternatives

The compliance engine reaches 96.53% raw and 96.78% corrected accuracy (95% CI 96.65–96.95) on 100,000 prompts (Section VII-A). That figure is not a head-to-head score against the systems surveyed in Section II, because none of them accepts the task as posed: an administrator-authored ruleset of 54 deployed policies, written in natural language with inline examples and exception clauses, applied to a prompt as a whole. Pattern-based DLP cannot express the exception clauses at all; its detectors act on the shape of a value, and the semantic gap described by Alneyadi et al. [5] — the format discussion and the live value share one shape — is exactly the material from which the benchmark's edge cases, masked records, and fixture values are built. Cloud content-safety APIs and Llama Guard [12] reason with models but evaluate fixed taxonomies with no PHI, GDPR, legal-privilege, or trade-secret category and no way for an administrator to add one. Guardrails frameworks presume the enterprise controls the LLM call site, which is not the case for the third-party tools the five enforcement surfaces intercept. The text-classification result of Hart et al. [8], $F_1 \in [0.90, 0.95]$ on narrow enterprise-email categories, is the closest published number for administrator-defined enterprise content, and it required a training corpus per category; the engine reaches its figure across 8 categories and 70 benchmark test facets from rule text, without a training corpus per policy. The reading we draw is therefore about coverage as much as accuracy: the engine holds a mid-90s accuracy on a task the alternatives do not attempt, and it does so with an open-weights model of roughly 20 billion parameters served locally, which is the under-evaluated regime that the LLM-as-judge literature (Section II-D) leaves open.

B. Why the Benign Pass Rate Matters More Than the Headline

For the administrator who decides whether the engine stays switched on, the operative number is not overall accuracy but the rate at which everyday work is blocked. Costante et al. [6] observe that false-positive volume is what drives administrators to disable DLP rules; a semantic engine inherits the same constraint. The benchmark separates that question from the headline. The benign band — 36,416 everyday prompts with no sensitive content — passes at 99.89%, with 40 false blocks. Specificity over all 60,977 pass-expected prompts, which include the deliberately hard masked, fixture, policy-question, and decoded-obfuscation material, is 96.59%; recall over the 38,982 scored violations is 96.43% (Section VII-D). The distance between 99.89% and 96.59% is where the false-positive mass lives: not in ordinary requests, but in pass-expected prompts constructed to look like violations. The error analysis in Section VII names the families — masked or placeholder values (542 misses), benign context (461), questions about policy (396), fixture values (324), decoded benign obfuscation (312) — and the deployed configuration already absorbs part of that mass through rule text alone: adding masked-record and fixture exception clauses moved the corrected figure from 96.16% to 96.78% on the same corpus, fixing 1,648 verdicts and changing 800 the other way. That is the lever an administrator holds. A false block in the benign band is what a user experiences; a false block on a fixture value is what a rule author can remove with an exception clause; and the two are reported separately so that a deployment can be planned against each.

C. The Semantic Gap the Architecture Closes

Pattern-based DLP fails when the signal that separates compliant from non-compliant content is semantic rather than surface-level [5], [6]. A detector that recognizes the shape of a Social Security Number flags 487-23-8891 in a violation and flags the same shape in a prompt that documents the format. The engine reads a rule with an explicit exception clause and decides on the surrounding intent, and it reads all 54 deployed policies at once, so a prompt nominally about one category is judged against every active rule in the same pass. The architecture in Section III closes the gap from both sides. Deterministic pre-analysis reverses the obfuscations that defeat a pattern detector — unicode and homoglyph folding, separator stripping, vertical layouts, base64 and hex decoding, leetspeak, spelled-out digits, announced reversals, split halves — and appends its findings to the evaluation input as data, never as instructions; the model then judges the canonical form against the rule. The adversarial band, 6,673 prompts across 13 techniques, scores 90.99%, and missed obfuscated values account for 192 of the 3,468 misses (Section VII). The mixed band, 9,920 realistic work prompts that pair a task request with sensitive or masked content, scores 92.25%; the multilingual band, 7,548 non-

English prompts across every category, scores 96.03%. What remains hard is the case with no lexical shape at all — a roadmap, an algorithm, an incident narrative, a set of board minutes, a contract clause — where the violation is the meaning of an ordinary paragraph; we return to it in Section XI-G.

D. Speed, and What Interactive Latency Enables

The benchmark measures speed on the same run that measures accuracy: 9.64 prompts/s sustained on two engines at 48 concurrent requests, about 4.8 prompts/s per engine, so that 100,000 prompts complete in roughly 2.9 hours (Section VII-H). At 16 concurrent requests the median allowed verdict returns in 1.10 s, the median blocked verdict in 2.69 s, and the overall p90 is 4.88 s; at 48 concurrent the medians are 2.18 s and 5.07 s. The forced final channel is what makes these numbers possible. A verdict written directly into the answer channel under a grammar-enforced schema costs about 71 generated tokens; the same verdict with full chain-of-thought costs about 249, 3.5× more, and the full-reasoning configuration runs at 4.08 prompts/s. Because the policy prefix of roughly 19,800 tokens is cached by the inference server, latency scales with output tokens rather than input length, which is why the allow path is fast and the block path — which writes a reason, cited rules, and a rewrite — is slower. Escalation preserves the arrangement: only about 3% of prompts take the additional full-reasoning call, at 4–10 s each, and a blocked verdict is never escalated, so the block path stays at forced-channel speed. The configuration comparison in Section VII shows the trade: the hybrid lands within 0.14 corrected points of full reasoning at 2.4× its throughput.

Interactive latency is what makes enforcement at the surface possible rather than advisory. A browser extension, desktop guard, or IDE extension holds the submit action while the verdict is pending; a sub-2-second median on allowed prompts means an ordinary request clears in the time a user expects a page to respond, and the compliant rewrite is produced inline for prompts under 2,000 characters, so a block on a short prompt can arrive with a redacted alternative rather than a bare refusal. For the infrastructure agent, which sits on the outbound path of server-side calls with no human waiting, the same throughput sets the request rate a single engine can guard. The 6-hour verdict cache, keyed on content, policy state, and engine configuration, serves repeated prompts without an engine call at all.

E. Fail-Closed as a Governance Property

An enforcement point that cannot reach a verdict has to choose between letting the prompt through and holding it. The engine holds: a verdict that hits the token ceiling, or a completion the parser cannot use, is returned as blocked with an explicit "compliance incomplete" reason, and only a genuinely unreachable engine follows the organization's configured fail-open or fail-closed setting. The benchmark tested the property rather than assuming it. The 100K run surfaced 41 prompts from one proprietary-algorithm template that returned "compliance unavailable" and were allowed; the captured raw completions showed the inference server ignoring a legacy schema-constraint field, the model looping on one rule reference at temperature 0 until the parser rejected the output, and the unparseable path inheriting fail-open as if it were an outage. After the fix — the current structured-output field, verified by replay, and a fail-closed rule for any unparseable completion — all 41 block, the 800 seeded controls are unchanged, and 0 of 100,000 prompts fall through to fail-open (Section VII). We treat this as a governance property in the same sense as the Admin Autonomy Property (Section III-I, validated in Section VII-G): each is a statement that enforcement follows the administrator's configuration and nothing else — not the model's pretraining, and not a parser's failure mode. The retest of 800 seeded control prompts on the deployed configuration — 399 of 400 true blocks held, 400 of 400 benign prompts held, none unavailable, at a median of 1.61 s — is the check that confirmed the fix without moving the controls, and the harness that produced it is versioned with the benchmark so that the check can be repeated (Section A).

F. Threats to Validity

Four threats bound the claims. The corpus is synthetic. Every prompt is generated under fixed seeds and validated for exact duplicates, balance, and length; the corpus is designed for coverage of 8 categories, 70 benchmark test facets, six bands, and 13 obfuscation techniques, not for representativeness of any organization's traffic. The benign band is the closest proxy for everyday use, and we report it separately for that reason, but we do not claim that 96.78% or 99.89% transfers to a random sample of production prompts. The ruleset belongs to one organization. The 54 deployed policies are one administrator-authored library; the 70 facets map many-to-one onto them, and 190 corpus labels were corrected before scoring because a reader audit of 400 random misses against the deployed rule texts found two clusters where the labels and the rules disagreed (a work email in a routing position; a public IP tied to a host). Another organization's rules would draw the boundary elsewhere, and the accuracy would move with it. Reader adjudication is itself a judgment. The corrected figure credits the engine for misses that eight independent readers, applying the deployed rule text, attributed to the dataset rather than the engine: 379 misses read, dataset share 18.0% [13.3, 23.9] among false negatives and 8.9% [5.6, 14.0] among false positives, Wilson intervals. Readers can be wrong in either direction; the protocol and their per-row outputs are versioned with the benchmark, we did not credit the 18 calls the readers marked arguable (crediting them would give 96.92%), and we report the raw 96.53% alongside the corrected figure. The interval is a statement about this corpus. The 95% CI of 96.65–96.95 is a property of

this corpus and this adjudication; it does not bound the engine on a different ruleset, a different language mix, or live traffic.

G. Limitations and Future Work

a) The remaining error mass is judgment-shaped.: Of 3,468 misses, the two largest families are semantic violations with no lexical shape — roadmaps, algorithms, incidents, board minutes, contracts — at 780 misses, 87% engine-attributable in the reader sample, and over-blocks on masked or placeholder values at 542, 95% engine-attributable. Together they are about 1.3 percentage points of accuracy, and they do not yield to prompt or rule-text engineering at forced-channel speed. The mechanical levers that remain are small: the canonicalizer families measured at +6 of the 192 obfuscated misses. The Confidential category, at 93.01% with 590 false negatives, carries the largest share of missed violations (Section VII-C).

b) The long band.: Prompts of 1,000–7,800 characters score 91.76%, and accuracy falls with length from 97.58% under 200 characters to 90.08% at 4,000–8,000 (Section VII-E). The band is violation-heavy by construction — buried payloads, split payloads, decoy safes — so it over-samples the hard block path; the benign long-form share allows at about 99%, and over-blocks on long benign documents (84 misses) were only 44% engine-attributable. The number to carry is that a needle buried in several thousand characters of ordinary prose is the engine's hardest block, not that long documents are blocked indiscriminately.

c) Multilingual semantic families.: The multilingual band scores 96.03%, above the core band's 95.89%, but its misses skew toward the same judgment-shaped family: 82 missed multilingual semantic violations, 71% engine-attributable. The band covers several European languages; other scripts are not in the corpus.

d) Why the reasoning re-judge was not adopted.: A re-judge with full chain-of-thought fixes about half of the masked-value over-blocks, and the full-reasoning configuration reaches 96.92% corrected. It costs 8–10 s per block, and interactive latency at the enforcement surface is a product requirement, so the hybrid escalates only the compliant verdicts that coincide with a value shape or a semantic-risk signal, and the block path never waits on reasoning. We accept a 0.14-point corrected gap in exchange for 2.4× the throughput.

e) Attribution drift under a disabled policy.: The Admin Autonomy re-validation (Section VII-G) found the forced channel enforcing switched-off content and labelling the block with the nearest enabled rule: 205 prompts containing only a card number stayed blocked with the card-number policy off. Neither rule-text scope sentences nor an explicit list of switched-off classes in the prompt moved it (194 of 195 and 188 of 194), and disabling the whole card-adjacent family only shifted the citations to a further ring of rules. The forced channel cannot be instructed out of this; it is the same judgment-shaped defect as the masked-value over-blocks. The remedy that holds is mechanical: a post-resolution guard that drops a block only when the text carries a switched-off policy's value shape and every cited rule is an identifier-class rule whose own shape is absent. Deployed, it allows 191 of 197 bare-card-number prompts with the policy off and keeps blocks whose cited content is genuinely present. Its limit is its table: semantic policies have no value shape, so a switched-off roadmap policy whose content the model files under board minutes is not caught. That residue belongs to the fine-tuning track, with the disable-and-probe experiment as its acceptance test.

f) Out of scope.: The adversarial band measures obfuscation of content a rule forbids, not prompt injection aimed at the engine's own system turn; we do not claim injection robustness (Section II-E). A cross-architecture comparison on identical policy sets is not run, for the reason given in Section XI-A.

g) Roadmap.: The path to the remaining error mass is supervised fine-tuning of the forced channel on adjudicated verdicts and full-reasoning teacher verdicts, with the benign band as the guardrail against trading false blocks for recall. With every engine-attributable defect fixed, the ceiling on this corpus is approximately 99.6%.

XII. Conclusion

We have presented a 100,000-prompt accuracy and efficiency benchmark of the deployed compliance engine of Whiteout AI: a locally-hosted, open-weights, approximately 20-billion-parameter model that reads 54 administrator-authored policies in its system turn and returns schema-constrained verdicts on a forced final channel, with deterministic pre-analysis before the judgment and hybrid escalation after it. Across 8 categories and six bands the engine achieves 96.53% raw and 96.78% corrected accuracy (95% CI [96.65%, 96.95%]), blocks 96.43% of violations, allows 96.59% of pass-expected prompts, and passes 99.89% of 36,416 benign everyday prompts. It does so at 9.64 prompts/s sustained on two engine instances, with a median latency of 1.10 s for allowed prompts.

The hybrid is what makes accuracy and speed compatible. On the same corpus and the same scoring, full chain-of-thought reasoning reaches 96.92% corrected accuracy at 4.08 prompts/s, and the forced channel alone reaches 95.02% at 10.73 prompts/s. The deployed configuration — forced channel with pre-analysis, escalation, and the masked-record and fixture exception clauses — recovers to within 0.14 corrected points of full reasoning at 2.4× its throughput, generating about 71 tokens per verdict instead of about 249. Only about 3% of prompts pay for a second, reasoned judgment, and blocked verdicts never do.

Enforcement is fail-closed by construction. A verdict that hits its token ceiling or cannot be parsed is held with an explicit "compliance incomplete" reason; only a genuinely unreachable engine follows the organization's

configured fail-open or fail-closed setting. The 100,000-prompt run surfaced and closed the one path that violated this rule: the 41 affected prompts now block, the 800 seeded controls are unchanged, and 0 of 100,000 prompts fall through to fail-open. The Admin Autonomy Property is now met mechanically for identifier-class policies: the forced channel could not be instructed to respect a switched-off policy, so a deterministic guard drops blocks that borrow a neighbouring rule's label for switched-off content, allowing 191 of 197 bare-card-number prompts with the card-number policy off (September 2026). Semantic policies, whose content has no value shape, remain an open item on the fine-tuning track; administrator configuration as the sole source of enforcement is the design goal the engine must meet in full.

The remaining error mass is judgment-shaped. Semantic violations with no lexical shape — roadmaps, algorithms, incident reports, board minutes, contracts — and over-blocks on masked or placeholder values together account for about 1.3 percentage points, and neither yields to prompt or rule-text engineering at forced-channel speed; the mechanical levers that remain are small, with additional canonicalizer families recovering 6 of the 192 obfuscated-value misses. Re-judging masked over-blocks with reasoning fixes about half of them but costs 8–10 s per block, and we did not adopt it, because interactive latency is a product requirement. The path forward is supervised fine-tuning of the forced channel on adjudicated verdicts and full-reasoning teacher verdicts, with the benign band as the guardrail against regressions on everyday prompts. With every engine-attributable defect fixed, the ceiling on this corpus is approximately 99.6%. Semantic compliance from a locally-hosted model reading natural-language rules is a deployable alternative to pattern-based DLP and fixed-taxonomy cloud content-safety APIs; the figures above are its measured shape today.

Data Availability

The corpus generators, the corpus validator and its committed manifest, the fixed seeds, the resumable concurrent runner, the adjudication protocol and reader outputs, and the retest harness with its seeded control prompts are versioned together; the corpus is released publicly on Hugging Face under Apache 2.0 (https://huggingface.co/datasets/ShmalexFlow/enterprise-ai-prompt-compliance-100k) and the tooling is available to partners on request. Section IV describes the corpus construction and Appendix A documents the artifacts.

Acknowledgments

The author thanks the Groovy Security engineering team for building and operating the platform whose compliance engine is evaluated here.

Appendix A
Reproducibility Appendix

The benchmark is reproducible exactly at the input layer and, with the engine configuration held fixed, at the verdict layer. This appendix lists the versioned artifacts that regenerate the corpus, re-run the evaluation, re-derive the corrected figure, and re-check the deployed engine, and it states their availability.

A. Fixed seeds and deterministic generation

The corpus generators are deterministic under fixed seeds: the 100,000 prompts regenerate from the generators and the recorded seeds, and the seeds are recorded in the manifest committed with the corpus (Section A-B). A regenerated corpus can therefore be checked against the one that was scored.

B. Corpus, validator, and manifest

The corpus comprises 100,000 prompts across 8 categories and 70 benchmark test facets, partitioned into six bands (core 36,095; long 3,348; adversarial 6,673; mixed 9,920; multilingual 7,548; benign 36,416), with 39,023 block-expected and 60,977 pass-expected labels. Each prompt carries its category, its policy_id (one of the 70 facets, mapping many-to-one onto the 54 deployed policies), its type (violation, safe, or edge_case), its expected verdict, its band, and its character length. A strict corpus validator runs over the generated set before it is accepted: it rejects exact duplicates and checks label balance and length distribution. The manifest committed with the corpus records how the corpus was produced, including the seeds. The 190 label corrections applied before scoring derive from the label audit described in Section IV; the reader outputs of that audit are versioned with the adjudication artifacts (Section A-D).

C. Generators, runner, and result records

Three classes of script regenerate and execute the benchmark. The corpus generators emit the six bands under the fixed seeds, and the validator accepts or rejects the result. The concurrent runner (Section IV-E) issues evaluation requests against the engine at a configured concurrency — 48 across two engines for the headline run — and is resumable: it checkpoints progress off-box, so an interrupted run continues from its last committed prompt rather than restarting, and a completed run's per-prompt records are the source of truth for every number in Section VII. The configuration comparison runs (Section VII) use the same runner and the same scoring over the same 100,000 prompts with only the engine configuration varied: full chain-of-thought reasoning; the forced final channel alone; the forced channel with escalation and pre-analysis; and the deployed configuration, which adds the masked-record and fixture exception clauses.

D. Adjudication protocol and reader outputs

The corrected figure is re-derivable from versioned inputs. Two mechanical classes are decided from rule text without readers: D1, the corporate-email carve-out (13 rows, credited to the dataset), and M1, a masked or placeholder value blocked despite an explicit rule exception (471 rows, charged to the engine). From the residual 3,174 misses the protocol draws 200 false negatives and 200 false positives at random, removes the rows the mechanical classes already decide (21), and gives the remaining 379 to eight independent readers together with the deployed rule text. A rule verdict that disagrees with the corpus label is a dataset error and is credited; a rule verdict that agrees with the label is an engine error and is not. The readers' per-row outputs, the 18 rows marked arguable (not credited), the resulting dataset shares (18.0% [13.3, 23.9] of false negatives and 8.9% [5.6, 14.0] of false positives, Wilson intervals), and the arithmetic from those shares to 96.78% are versioned together, as are the outputs of the earlier label audit (400 random misses; 190 corrections).

E. Retest harness and seeded controls

The retest harness carries 800 seeded control prompts — 400 true blocks and 400 benign prompts — and runs them against the deployed configuration. Its result on the deployed configuration is 399 of 400 true blocks held, 400 of 400 benign held, 0 unavailable, at a median latency of 1.61 s. The same controls were used to verify the fail-closed fix described in Section VII: the 41 previously fail-open prompts all block, the 800 controls are unchanged, and 0 of the 100,000 corpus prompts fall through to fail-open.

F. Engine configuration

A reproduction must hold the engine configuration fixed, because verdicts and the verdict cache are keyed on it. The configuration comprises: the 54 deployed policies as rule text, placed in the system turn (a prefix of roughly 19,800 tokens, prefix-cached by the inference server); the deterministic pre-analysis — identifier pre-detector and canonicalizer — whose findings are appended to the evaluation input as data; the forced final channel with grammar-enforced structured decoding against the verdict schema (status, reason, cited rules, optional rewrite), using the inference server's current structured-output field; the escalation triggers (pre-analysis value shapes and the semantic-risk signals listed in Section III); the fail-closed rule for token-ceiling and unparseable completions; the chunking threshold of 8,000 characters and the inline-rewrite threshold of 2,000 characters; and decoding at temperature 0. The model is an open-weights, roughly 20-billion-parameter mixture-of-experts model post-trained for policy-conditioned classification, served locally on a single GPU under the organization's control. The headline run used two engines at 48 concurrent requests; latency is reported at both 16 and 48 concurrent requests.

G. Availability

The corpus generators, the validator, the manifest and fixed seeds, the concurrent runner and its checkpoints, the adjudication protocol and reader outputs, the retest harness with its seeded controls, and the configuration comparison runs are versioned together. The corpus is released publicly on Hugging Face under Apache 2.0 (https://huggingface.co/datasets/ShmalexFlow/enterprise-ai-prompt-compliance-100k), as a single parquet file with a dataset card describing the bands, schema, label provenance, and reference result; the remaining artifacts are available to partners on request.

H. What is and is not reproducible

Reproducible exactly. The corpus and its labels (from the generators, seeds, and manifest, plus the 190 corrections), the per-prompt result records of the headline and comparison runs, the mechanical adjudication classes, the reader outputs, and every statistic derived from those records, including the raw and corrected figures and their intervals.

Reproducible with the configuration held fixed. The engine's verdicts on a fresh run. Decoding is at temperature 0 and the output is schema-constrained, but a verdict depends on the full configuration above; a change to the rule text, the pre-analysis, the escalation triggers, or the structured-output enforcement changes verdicts, which is what the configuration comparison measures and what the retest harness guards.

Not reproducible from this report alone. The latency and throughput figures, which depend on the hardware and serving stack behind the two engines and on concurrent load. A reproduction should expect the ordering to hold — allowed verdicts faster than blocked, the forced channel faster than full reasoning, escalations rare — and the absolute values to move with the hardware.

References

[1] European Data Protection Board, "Opinion 28/2024 on certain data protection aspects related to the processing of personal data in the context of AI models," https://www.edpb.europa.eu/our-work-tools/our-documents/opinion-board-art-64/opinion-282024-certain-data-protection-aspects_en, 2024.

[2] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, A. Oprea, and C. Raffel, "Extracting training data from large language models," in 30th USENIX Security Symposium, 2021, pp. 2633–2650.

[3] M. Nasr, N. Carlini, J. Hayase, M. Jagielski, A. F. Cooper, D. Ippolito, C. A. Choquette-Choo, E. Wallace, F. Tramer, and K. Lee, "Scalable extraction of training data from (production) language models," arXiv preprint arXiv:2311.17035, 2023.

[4] A. Shabtai, Y. Elovici, and L. Rokach, A Survey of Data Leakage Detection and Prevention Solutions, ser. SpringerBriefs in Computer Science. Springer, 2012.

[5] S. Alneyadi, E. Sithirasenan, and V. Muthukkumarasamy, "A survey on data leakage prevention systems," Journal of Network and Computer Applications, vol. 62, pp. 137–152, 2016.

[6] E. Costante, D. Fauri, S. Etalle, J. den Hartog, and N. Zannone, "A hybrid framework for data loss prevention and detection," in 2016 IEEE Security and Privacy Workshops (SPW), 2016, pp. 324–333.

[7] B. Hauer, "Data and information leakage prevention within the scope of information security," IEEE Access, vol. 3, pp. 2554–2565, 2015.

[8] M. Hart, P. Manadhata, and R. Johnson, "Text classification for data loss prevention," in Privacy Enhancing Technologies (PETS 2011), ser. Lecture Notes in Computer Science, vol. 6794. Springer, 2011, pp. 18–37.

[9] T. Markov, C. Zhang, S. Agarwal, F. E. Nekoul, T. Lee, S. Adler, A. Jiang, and L. Weng, "A holistic approach to undesired content detection in the real world," in Proceedings of the AAAI Conference on Artificial Intelligence, 2023, arXiv:2208.03274.

[10] A. Lees, V. Q. Tran, Y. Tay, J. Sorensen, J. Gupta, D. Metzler, and L. Vasserman, "A new generation of Perspective API: Efficient multilingual character-level transformers," in Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22), 2022, pp. 3197–3207.

[11] M. Sap, D. Card, S. Gabriel, Y. Choi, and N. A. Smith, "The risk of racial bias in hate speech detection," in Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019, pp. 1668–1678.

[12] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, "Llama Guard: LLM-based input-output safeguard for human-ai conversations," arXiv preprint arXiv:2312.06674, 2023.

[13] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2023, pp. 431–445, arXiv:2310.10501.

[14] Guardrails AI Project, "Guardrails AI: Adding guardrails to large language models," https://github.com/guardrails-ai/guardrails, 2024.

[15] Y. Dong, R. Mu, G. Jin, Y. Qi, J. Hu, X. Meng, X. Zhao, and X. Huang, "Safeguarding large language models: A survey," arXiv preprint arXiv:2406.02622, 2024.

[16] D. Kang, X. Li, I. Stoica, C. Guestrin, M. Zaharia, and T. Hashimoto, "Exploiting programmatic behavior of LLMs: Dual-use through standard security attacks," in 2024 IEEE Security and Privacy Workshops (SPW), 2024, arXiv:2302.05733.

[17] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in Advances in Neural Information Processing Systems 36 (NeurIPS 2023) Datasets and Benchmarks Track, 2023, arXiv:2306.05685.

[18] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion et al., "Constitutional AI: Harmlessness from AI feedback," arXiv preprint arXiv:2212.08073, 2022.

[19] C.-H. Chiang and H.-y. Lee, "Can large language models be an alternative to human evaluations?" in Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), 2023.

[20] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG evaluation using GPT-4 with better human alignment," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023.

[21] P. Wang, L. Li, L. Chen, D. Zhu, B. Lin, Y. Cao, L. Kong, Q. Liu, T. Liu, and Z. Sui, "Large language models are not fair evaluators," arXiv preprint arXiv:2305.17926, 2023.

[22] Y. Dubois, B. Galambosi, P. Liang, and T. B. Hashimoto, "Length-controlled AlpacaEval: A simple way to debias automatic evaluators," arXiv preprint arXiv:2404.04475, 2024.

[23] R. Li, T. Patel, F. Viegas, M. Wattenberg et al., "Benchmarking cognitive biases in large language models as evaluators," arXiv preprint arXiv:2309.17012, 2024.

[24] J. Gu, B. Bebensee, M. Liu, W. Fan, X. Zhao, and Y. Wang, "A survey on LLM-as-a-judge," arXiv preprint arXiv:2411.15594, 2024.

[25] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in NeurIPS 2022 ML Safety Workshop, 2022, arXiv:2211.09527.

[26] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), 2023.

[27] A. Wei, N. Haghtalab, and J. Steinhardt, "Jailbroken: How does LLM safety training fail?" in Advances in Neural Information Processing Systems 36 (NeurIPS 2023), 2023.

[28] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," arXiv preprint arXiv:2307.15043, 2023.

[29] N. Jain, A. Schwarzschild, Y. Wen, G. Somepalli, J. Kirchenbauer, P.-Y. Chiang, M. Goldblum, A. Saha, J. Geiping, and T. Goldstein, "Baseline defenses for adversarial attacks against aligned language models," arXiv preprint arXiv:2309.00614, 2023.

[30] M. Mazeika, L. Phan, X. Yin, A. Zou, Z. Wang, N. Mu, E. Sakhaee, N. Li, S. Basart, B. Li, D. Forsyth, and D. Hendrycks, "HarmBench: A standardized evaluation framework for automated red teaming and robust refusal," in Proceedings of the 41st International Conference on Machine Learning (ICML), 2024.

[31] P. Chao, E. Debenedetti, A. Robey, M. Andriushchenko, F. Croce, V. Sehwag, E. Dobriban, N. Flammarion, G. J. Pappas, F. Tramer, H. Hassani, and E. Wong, "JailbreakBench: An open robustness benchmark for jailbreaking large language models," in Advances in Neural Information Processing Systems 37 (NeurIPS 2024) Datasets and Benchmarks Track, 2024, arXiv:2404.01318.

[32] X. Pan, M. Zhang, S. Wang, Y. Yang, and M. Wang, "Privacy risks of general-purpose language models," in 2020 IEEE Symposium on Security and Privacy (SP), 2020, pp. 1314–1331.

[33] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," in International Conference on Learning Representations (ICLR), 2021.